

Титульный лист

Содержание

Введение.....	4
1. Описание мобильной платформы.....	5
2. Математическая модель и синтез закона управления мобильной платформой	10
2.1. Кинематическая модель мобильной платформы.....	11
2.2. Постановка задачи управления.....	14
2.3. Синтез закона управления мобильной платформой.....	17
2.4. Выводы.....	22
3. Разработка программы.....	24
3.1. Назначение программы.....	24
3.2. Функциональные требования к программе.....	25
3.3. Архитектура программы.....	29
3.3.1. Подсистема управляющей логики.....	31
3.3.2. Вычислительная подсистема управления.....	32
3.3.3. Подсистема аппаратной абстракции.....	34
3.3.4. Платформенно-зависимая подсистема.....	35
3.3.5. Взаимодействие подсистем в цикле управления.....	36
3.3.6. Выводы.....	37
3.4. Программная реализация цикла управления.....	38
3.4.1. Общий шаг управления.....	39
3.4.2. Обновление датчиков и формирование текущего состояния платформы.....	41
3.4.3. Вычисление управляющего воздействия.....	44
3.4.4. Ограничение и передача управляющего воздействия исполнительным механизмам.....	46
3.4.5. Вспомогательные процедуры обеспечения устойчивости и безопасной работы.....	48
3.4.6. Связь реализованных алгоритмов с архитектурой системы.....	49
3.4.7. Выводы.....	50
3.5. Тестирование.....	50
3.5.1. Модульное тестирование.....	52
3.5.2. Тестирование в среде моделирования.....	55
3.5.2.1. Интеграция системы управления со средой моделирования.....	56
3.5.2.2. Методика экспериментальной оценки.....	57
3.5.2.3. Результаты тестирования.....	60
3.5.2.3.1. Прямолинейное движение.....	61
3.5.2.3.2. Поворот на месте.....	63
3.5.2.3.3. Движение по окружности.....	66
3.5.2.3.4. Движение по квадратной траектории.....	69
3.5.2.4. Выводы по испытаниям в среде моделирования.....	72
3.6. Выводы.....	73
4. Особенности реализации на платформе STM32.....	75
4.1 Общие положения.....	75

4.2 Признаки ориентации проекта на платформу STM32.....	76
4.3 Требования к целевой платформе.....	79
4.4 Соответствие архитектуры программы требованиям платформы STM32	80
4.5 Реализация взаимодействия с аппаратными ресурсами.....	82
4.6 Организация цикла управления на микроконтроллере.....	84
4.7 Особенности реализации и ограничения перехода от моделирования к целевой платформе.....	86
4.8 Выводы.....	88
Заключение.....	90
Список источников.....	91
Приложения.....	92
Приложение А.....	93
Приложение Б.....	94
Приложение В.....	95

Введение

Развитие мобильной робототехники тесно связано с созданием встраиваемых программных систем, обеспечивающих формирование и поддержание требуемого режима движения робототехнических платформ. Для мобильного робота качество работы ходовой части определяет возможность устойчивого перемещения, выполнения манёвров, соблюдения заданных параметров траектории и последующей интеграции с более высокими уровнями управления. В связи с этим задача разработки программ управления движением представляет собой одну из ключевых задач при создании робототехнических систем [11, 16].

Особую практическую значимость имеют программные решения, ориентированные на использование во встраиваемых вычислительных системах класса микроконтроллеров [9, 15, 21]. Такие системы должны обеспечивать не только корректность вычисления управляющих воздействий, но и возможность их детерминированного выполнения в условиях ограниченных вычислительных ресурсов, конечного объёма памяти и необходимости непосредственного взаимодействия с датчиками и исполнительными механизмами. В этом смысле разработка программы управления ходовой частью мобильного робота для платформы STM32 является актуальной задачей, сочетающей вопросы математического моделирования, синтеза закона управления, архитектурного проектирования программного обеспечения и его последующей адаптации к целевой аппаратной среде [1, 10, 12, 15, 21].

В качестве объекта управления в работе рассматривается четырёхколёсная мобильная платформа с дифференциальной схемой привода. Такая платформа обладает сравнительно простой механической структурой, удобна для математического описания и в то же время сохраняет основные свойства реального объекта автоматического управления, включая наличие погрешностей измерения, отклонений параметров модели от поведения реальной системы и

необходимость использования обратной связи для компенсации возникающих отклонений [1, 5, 8, 11].

Актуальность темы работы обусловлена тем, что при разработке программного обеспечения для мобильных роботов требуется обеспечить согласование нескольких взаимосвязанных требований [5, 8, 11]. С одной стороны, система управления должна корректно преобразовывать задаваемые линейную и угловую скорости платформы в управляющие воздействия на приводы. С другой стороны, программная реализация должна быть архитектурно организована таким образом, чтобы вычислительная логика не зависела от конкретной аппаратной платформы и могла быть проверена в моделирующей среде до этапа натурной интеграции [10, 12, 13]. Наконец, сама программа должна быть ориентирована на последующий перенос на микроконтроллерную платформу STM32, что требует выделения платформенно-зависимых и платформенно-независимых компонентов системы [9, 15, 20, 21, 22].

Объектом исследования является мобильная робототехническая платформа с дифференциальной схемой привода.

Предметом исследования является программная система управления ходовой частью мобильной платформы, обеспечивающая формирование управляющих воздействий на приводы по заданным параметрам движения с использованием математической модели и обратной связи.

Целью выпускной квалификационной работы является разработка программы управления ходовой частью четырёхколёсного мобильного робота, обеспечивающей воспроизведение заданного режима движения платформы и ориентированной на последующую реализацию на платформе STM32.

Для достижения поставленной цели в работе решаются следующие задачи:

1. выполнить описание мобильной платформы как объекта управления и выделить её конструктивные и функциональные особенности, существенные для построения системы управления;

2. выбрать и построить математическую модель объекта управления, адекватную поставленной задаче формирования движения платформы;
3. сформулировать задачу управления и разработать закон управления, обеспечивающий соответствие фактических параметров движения заданным;
4. спроектировать архитектуру программной системы управления с разделением вычислительной логики, аппаратной абстракции и платформенно-зависимых компонентов;
5. реализовать программный дискретный цикл управления, включающий получение данных датчиков, формирование текущего состояния платформы, вычисление управляющего воздействия и его передачу исполнительным механизмам;
6. провести модульное тестирование разработанных компонентов и выполнить испытания полной системы в среде моделирования;
7. проанализировать особенности адаптации и переноса разработанной программы на целевую платформу STM32.

В работе используются методы математического моделирования, элементы теории автоматического управления, методы объектно-ориентированного проектирования программного обеспечения, а также методы модульного и интеграционного тестирования [1, 7, 10, 12, 19]. Проверка корректности функционирования разработанной программы выполняется в среде моделирования, где исследуется работа полного замкнутого контура управления в характерных режимах движения мобильной платформы [13, 18].

Практическая значимость работы состоит в том, что в ней разработана программная основа системы управления ходовой частью мобильного робота, в которой вычислительное ядро отделено от платформенно-зависимой реализации взаимодействия с аппаратными ресурсами [10, 12]. Такое построение позволяет использовать единую вычислительную логику как в моделирующей среде, так и при последующем переносе на платформу STM32 [9, 13, 15, 21, 22].

Практическая ценность работы заключается в возможности дальнейшего использования разработанной программы в составе реальной встраиваемой системы управления мобильной платформой.

Структура работы определяется поставленной целью и логикой исследования. В первой главе приводится описание мобильной платформы как объекта управления. Во второй главе рассматриваются построение математической модели и синтез закона управления. В третьей главе приводятся назначение программы, функциональные требования, архитектура, программная реализация цикла управления и результаты тестирования. В четвёртой главе анализируются особенности переноса разработанной программы на платформу STM32.

1. Описание мобильной платформы

В качестве объекта управления в работе рассматривается мобильная робототехническая платформа, предназначенная для перемещения в плоскости с заданными параметрами поступательного и вращательного движения. С функциональной точки зрения данная платформа представляет собой механическую систему, способную изменять своё положение и ориентацию под действием управляющих воздействий, формируемых программной системой управления ходовой частью. Следовательно, в рамках настоящей работы робот рассматривается как объект автоматического управления, для которого требуется обеспечить воспроизведение заданного режима движения [1, 16].

Конструктивно рассматриваемая платформа выполнена по четырёхколёсной схеме. В состав ходовой части входят два ведущих колеса, расположенных по бокам корпуса, и два опорных роликовых колеса, размещённых в передней и задней частях платформы. Ведущие колёса являются активными исполнительными элементами и непосредственно участвуют в формировании движения робота. Опорные роликовые колёса собственного привода не имеют и выполняют вспомогательную функцию, связанную с обеспечением устойчивости конструкции, поддержанием корпуса относительно опорной поверхности и перераспределением нагрузки [16].

С точки зрения кинематики рассматриваемая платформа относится к классу мобильных роботов с дифференциальной схемой привода. Особенность данной схемы состоит в том, что движение платформы в плоскости определяется независимым управлением двумя ведущими колёсами. Иначе говоря, общее движение робота формируется как результат совместной работы двух приводов, а изменение режима движения достигается изменением скоростей вращения левого и правого колёс. Такая схема широко применяется в мобильной робототехнике благодаря конструктивной простоте, удобству математического описания и возможности реализации основных режимов движения без усложнения механической структуры объекта [5, 8, 11].

Физический смысл движения при дифференциальной схеме привода определяется соотношением между скоростями ведущих колёс. Если скорости левого и правого колеса совпадают по величине и направлению, платформа движется прямолинейно. Если скорости различаются, движение становится криволинейным, поскольку возникает вращение корпуса относительно мгновенного центра поворота. Если же скорости колёс равны по модулю, но противоположны по знаку, реализуется разворот на месте без поступательного перемещения платформы. Таким образом, изменение характера движения достигается без изменения геометрии ходовой части, исключительно за счёт изменения относительных скоростей двух исполнительных каналов [5, 8, 11].

Выбор именно дифференциальной схемы привода в качестве основы рассматриваемой платформы обусловлен её конструктивной простотой, удобством инженерного анализа и естественным соответствием задаче разработки программной системы управления ходовой частью. Такая платформа допускает переход от команд верхнего уровня, задаваемых в терминах линейной и угловой скоростей, к управляющим воздействиям на конкретные исполнительные механизмы. По этой причине она является удобным объектом для построения математической модели и последующего синтеза закона управления [5, 8, 11].

С позиций теории автоматического управления рассматриваемая платформа представляет собой управляемую механическую систему, состояние которой изменяется под действием входных воздействий. Входами объекта управления являются управляющие сигналы, подаваемые на приводы ведущих колёс. В зависимости от уровня абстракции эти воздействия могут интерпретироваться как сигналы на двигатели, как требуемые угловые скорости вращения колёс либо как обобщённые линейная и угловая скорости платформы. Выходами системы являются измеряемые или оцениваемые параметры движения, к которым относятся угловые скорости ведущих колёс, линейная и

угловая скорости платформы, а также её положение и ориентация в плоскости [1, 11].

В рамках настоящей работы корпус платформы рассматривается как абсолютно жёсткое тело, движение которого ограничено плоскостью. Такое допущение позволяет не учитывать вертикальные перемещения, наклоны корпуса, деформации конструкции и другие пространственные эффекты, несущественные для поставленной задачи. При этом положение платформы может быть описано координатами характерной точки корпуса в плоскости и углом ориентации относительно выбранного направления отсчёта. Подобная постановка соответствует принятому далее кинематическому описанию объекта управления [8, 11].

Принципиально важно, что в рассматриваемой платформе активными исполнительными звеньями ходовой части являются только два ведущих колеса. Опорные роликовые элементы не участвуют в создании тягового усилия и не формируют собственного управляющего воздействия на движение робота. Их влияние на поведение платформы учитывается косвенно — через предположение о сохранении устойчивого контакта с опорной поверхностью. Это позволяет в дальнейшем сосредоточить математическое описание на связи между движением ведущих колёс и движением платформы как единого объекта [5, 8].

С инженерной точки зрения рассматриваемая платформа не может считаться полностью детерминированной системой, поведение которой во всех режимах в точности совпадает с номинальным описанием. На её движение оказывают влияние погрешности измерений, неидеальность приводов, отклонение параметров реального объекта от расчётных значений, а также внешние возмущения. Следовательно, даже при использовании сравнительно простой модели движение платформы требует корректирующего воздействия, формируемого по обратной связи. Это обстоятельство имеет принципиальное значение для дальнейшего построения системы управления, в которой модельное

описание используется для задания номинального режима движения, а регулятор — для компенсации отклонений от него [1, 6, 14].

Для дальнейшего построения алгоритма управления существенно, что движение платформы может быть описано через обобщённые параметры — линейную и угловую скорости. Такое представление удобно как с точки зрения математического моделирования, так и с точки зрения программной реализации, поскольку позволяет верхнему уровню системы управления оперировать режимом движения платформы как единого объекта, а не параметрами каждого колеса по отдельности. Именно это представление используется в последующих разделах работы при постановке задачи управления и разработке программной реализации вычислительного контура [5, 8, 11].

В рамках данной работы для описания движения платформы используется кинематическая модель. Такой подход означает, что основное внимание уделяется связи между параметрами движения ведущих колёс и геометрией движения платформы без явного учёта сил, моментов и инерционных эффектов. Выбор именно кинематического описания обусловлен характером решаемой задачи, связанной с разработкой программной системы управления движением на уровне скоростных команд. Использование полной динамической модели в данной постановке не является необходимым, поскольку привело бы к существенному усложнению математического описания и алгоритмической реализации без принципиального выигрыша для рассматриваемого класса задач [8, 11].

Следует, однако, отметить, что кинематическое описание опирается на ряд упрощающих предположений. Предполагается отсутствие существенного проскальзывания ведущих колёс относительно опорной поверхности, постоянство геометрических параметров платформы, движение по ровной поверхности и пренебрежение рядом эффектов динамической природы, способных повлиять на фактическое поведение системы. Такие допущения допустимы для синтеза базового закона управления, однако их приближённый

характер означает, что математическая модель не может полностью совпадать с реальным объектом во всех режимах работы. Поэтому при практической реализации системы управления требуется использование обратной связи, компенсирующей отклонения, не учитываемые номинальной моделью [1, 8, 11].

Общий вид рассматриваемой мобильной платформы и расположение основных элементов её ходовой части представлены на рисунке 1.

[тут я вставляю рисунок когда нарисую его]

Рисунок 1 — Общий вид мобильной платформы с дифференциальной схемой привода

Представленная конструктивная схема показывает, что активное формирование движения платформы осуществляется двумя ведущими колёсами, тогда как остальные элементы ходовой части выполняют вспомогательные функции. Такое устройство платформы определяет выбор её кинематического описания и служит основанием для дальнейшего построения математической модели объекта управления.

Таким образом, рассматриваемая мобильная платформа представляет собой типичный, но в то же время достаточно содержательный объект управления для задач разработки программного обеспечения ходовой части. Она допускает сравнительно простое математическое описание, естественно согласуется с представлением движения в терминах линейной и угловой скоростей и позволяет реализовать программную систему управления, в которой математическая модель используется для формирования номинального режима движения, а обратная связь — для компенсации отклонений от него [1, 5, 8, 11]. Детальное математическое описание объекта и синтез закона управления рассматриваются в следующей главе.

2. Математическая модель и синтез закона управления мобильной платформой

Разработка программы управления ходовой частью мобильного робота требует предварительного математического описания объекта управления и формализации требований к его движению. Без такого описания невозможно обоснованно определить состав входных и выходных переменных системы, установить взаимосвязь между задаваемыми параметрами движения и фактически реализуемыми воздействиями на приводы, а также построить закон управления, обеспечивающий воспроизведение требуемого режима движения [1].

В рамках настоящей работы в качестве объекта управления рассматривается мобильная платформа с дифференциальной схемой привода, конструктивные особенности которой были приведены в предыдущей главе. В качестве математического описания объекта управления используется кинематическая модель мобильной платформы. Выбор именно кинематической модели обусловлен тем, что решаемая задача связана с формированием режима движения платформы по заданным линейной и угловой скоростям, а не с детальным исследованием силовых процессов в приводах и механической системе. При принятых допущениях о движении по плоской поверхности, отсутствии существенного бокового проскальзывания и работе в режимах, для которых определяющими являются кинематические ограничения, такая модель позволяет установить однозначную связь между параметрами движения платформы и угловыми скоростями ведущих колёс при существенно меньшей сложности по сравнению с полной динамической моделью [8, 11]. Дополнительным фактором в пользу данного выбора является малая масса платформы, вследствие чего в рамках рассматриваемой задачи влияние сложных инерционных эффектов можно считать второстепенным.

Следует отметить, что использование только номинальной математической модели не позволяет в полном объёме учесть реальные условия

функционирования системы. На движение мобильной платформы оказывают влияние погрешности задания параметров, неточность коэффициентов модели, особенности приводов и внешние возмущающие воздействия. По этой причине в работе применяется комбинированный подход к построению управления: номинальные управляющие значения определяются на основе математической модели, а отклонения фактического движения от требуемого компенсируются с помощью обратной связи. Такая структура позволяет совместить простоту модельного расчёта с повышением точности управления в условиях реальной эксплуатации [1, 6].

2.1. Кинематическая модель мобильной платформы

При построении математической модели принимаются следующие допущения: платформа рассматривается как абсолютно твёрдое тело, движение происходит по плоской поверхности, боковое проскальзывание отсутствует, а радиусы ведущих колёс и геометрические параметры платформы считаются постоянными. Указанные допущения позволяют использовать кинематическую модель, на основе которой определяются требуемые скорости ведущих колёс, используемые далее при формировании номинальной составляющей управления и её последующей коррекции по обратной связи [8, 11].

Положение платформы в неподвижной системе координат определяется координатами характерной точки платформы и углом её ориентации. В качестве вектора состояния примем

$$q(t) = \begin{pmatrix} x(t) \\ y(t) \\ \theta(t) \end{pmatrix}$$

где $x(t)$ и $y(t)$ — координаты характерной точки платформы в плоскости движения, а $\theta(t)$ — угол ориентации продольной оси платформы относительно неподвижной системы координат [11].

Для описания движения рассмотрим две системы координат: неподвижную глобальную систему координат, связанную с рабочим

пространством, и локальную систему координат, связанную с платформой. В локальной системе координат поступательное движение платформы направлено вдоль её продольной оси, а вращательное движение характеризуется угловой скоростью вокруг вертикальной оси. Если линейная скорость платформы вдоль продольной оси равна $v(t)$, а угловая скорость — $\omega(t)$, то переход к неподвижной системе координат даёт следующие кинематические уравнения [8, 11]:

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

Первое уравнение показывает, что изменение координаты x определяется проекцией линейной скорости платформы на ось абсцисс неподвижной системы координат. Аналогично, второе уравнение задаёт изменение координаты y как проекцию линейной скорости на ось ординат. Третье уравнение отражает изменение ориентации платформы во времени и непосредственно задаётся её угловой скоростью.

Таким образом, движение платформы полностью определяется двумя величинами: линейной скоростью $v(t)$ и угловой скоростью $\omega(t)$. Однако в реальной системе эти величины не задаются непосредственно исполнительным механизмом. Управляющее воздействие формируется через приводы левого и правого ведущих колёс, поэтому необходимо установить связь между скоростями платформы как целого и скоростями отдельных колёс [5, 8, 11].

Обозначим через $v_L(t)$ и $v_R(t)$ линейные скорости точек контакта левого и правого ведущих колёс с опорной поверхностью, а через L — расстояние между ведущими колёсами. При прямолинейном движении платформы скорости обоих колёс совпадают, и платформа движется без изменения ориентации. При различии скоростей колёс возникает вращение платформы. В рамках кинематики дифференциального привода линейная и угловая скорости платформы определяются выражениями [5, 8, 11]:

$$v = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{L}$$

Первое соотношение означает, что линейная скорость платформы равна средней линейной скорости ведущих колёс. Это следует из того, что при отсутствии бокового проскальзывания общее поступательное движение формируется совместным действием обоих приводов. Второе соотношение показывает, что угловая скорость платформы пропорциональна разности скоростей правого и левого колёс: чем больше эта разность, тем интенсивнее поворот платформы [5, 8].

Для последующего решения задачи управления требуется также обратное преобразование, то есть определение скоростей колёс по заданным линейной и угловой скоростям платформы. Решая приведённую систему относительно $v_L(t)$ и $v_R(t)$, получаем [5, 8, 11]:

$$v_L = v - \frac{L\omega}{2}$$

$$v_R = v + \frac{L\omega}{2}$$

Эти выражения имеют очевидный физический смысл. При положительной угловой скорости $\omega(t)$ платформа должна поворачиваться в одну сторону, что достигается уменьшением скорости одного колеса и увеличением скорости другого. При $\omega(t)=0$ обе скорости становятся равными, и движение происходит прямолинейно. При $v(t)=0$ и ненулевой $\omega(t)$ формируется режим разворота на месте, когда колёса вращаются в противоположных направлениях [5].

Поскольку в программной реализации и при работе с приводами удобно использовать угловые скорости вращения колёс, линейные скорости $v_L(t)$ и $v_R(t)$ дополнительно выражаются через угловые скорости $\omega_L(t)$ и $\omega_R(t)$. Пусть r — радиус ведущего колеса. Тогда [5, 8]:

$$\omega_L(t) = \frac{v_L(t)}{r},$$

$$\omega_R(t) = \frac{v_R(t)}{r}.$$

Подставляя сюда найденные ранее выражения для линейных скоростей колёс, получаем [5, 8]

$$\omega_L = \frac{1}{r} \left(v - \frac{L\omega}{2} \right)$$

$$\omega_R = \frac{1}{r} \left(v + \frac{L\omega}{2} \right)$$

Полученные зависимости являются ключевыми для дальнейшего построения системы управления. Они устанавливают однозначную связь между требуемыми параметрами движения платформы и угловыми скоростями вращения её ведущих колёс. Иными словами, кинематическая модель позволяет перейти от описания движения платформы как единого объекта к формированию опорных значений для каждого привода, которые в дальнейшем используются при синтезе закона управления [5, 8, 11].

2.2. Постановка задачи управления

После построения кинематической модели объекта управления необходимо формализовать задачу управления, то есть определить задающие и регулируемые величины, а также требуемый результат функционирования системы [1].

В рамках рассматриваемой системы на вход программы управления поступают заданные значения линейной и угловой скоростей платформы

$$u(t) = \begin{pmatrix} v_{зад}(t) \\ \omega_{зад}(t) \end{pmatrix}$$

Указанные сигналы определяют требуемый режим движения мобильной платформы в плоскости. Заданная линейная скорость характеризует интенсивность поступательного движения, а заданная угловая скорость —

интенсивность изменения ориентации платформы. Совокупность этих величин полностью определяет желаемое кинематическое поведение объекта управления в каждый момент времени [8, 11].

Однако непосредственное управление исполнительными механизмами по величинам $v(t)$ и $\omega(t)$ невозможно, поскольку приводы воздействуют не на платформу как на единый объект, а на её отдельные ведущие колёса. Поэтому первой частью задачи управления является преобразование заданных параметров движения платформы в параметры движения ведущих колёс. На основе кинематической модели для дифференциальной платформы требуемые угловые скорости левого и правого ведущих колёс определяются выражениями [5, 8]:

$$\omega_{L, \text{зад}}(t) = \frac{1}{r} \left(v_{\text{зад}}(t) - \frac{L \omega_{\text{зад}}(t)}{2} \right)$$
$$\omega_{R, \text{зад}}(t) = \frac{1}{r} \left(v_{\text{зад}}(t) + \frac{L \omega_{\text{зад}}(t)}{2} \right)$$

Указанное преобразование необходимо для формирования номинального режима работы приводов, соответствующего заданному движению платформы. Однако в текущей реализации системы управления обратная связь организована не по угловым скоростям отдельных колёс, а по линейной и угловой скоростям платформы как единого объекта. Такой подход соответствует общей структуре управления движением, в которой объектом регулирования выступает кинематическое состояние платформы в целом, а не каждый привод изолированно [1, 11].

Фактические линейная и угловая скорости платформы определяются на основе данных энкодеров ведущих колёс. При этом фактическая линейная скорость платформы обозначается через $v(t)$, а фактическая угловая скорость — через $\omega(t)$. Ошибки регулирования определяются как отклонения фактических скоростей платформы от их заданных значений [1, 6]:

$$e_v(t) = v_{\text{зад}}(t) - v(t)$$

$$e_\omega(t) = \omega_{\text{зад}}(t) - \omega(t)$$

Эти выражения отражают степень отклонения реального движения платформы от требуемого режима. Если ошибки равны нулю, фактическое движение платформы полностью соответствует задаваемым параметрам. При ненулевых значениях ошибок система управления должна формировать корректирующее воздействие, направленное на уменьшение рассогласования [1].

Следовательно, задача управления в рассматриваемой работе формулируется следующим образом: требуется построить такой закон управления, при котором фактические линейная и угловая скорости мобильной платформы $v(t)$ и $\omega(t)$ стремятся к соответствующим заданным значениям $v_{\text{зад}}(t)$ и $\omega_{\text{зад}}(t)$. Формально цель управления может быть записана в виде [1]:

$$\lim_t v(t) = v_{\text{зад}}(t)$$

$$\lim_t \omega(t) = \omega_{\text{зад}}(t)$$

Поскольку линейная и угловая скорости платформы однозначно связаны с угловыми скоростями ведущих колёс, выполнение указанных условий означает воспроизведение требуемого режима движения мобильной платформы в целом. Тем самым кинематическая модель используется для вычисления номинальных параметров движения колёс, а обратная связь — для обеспечения соответствия фактического движения платформы заданным линейной и угловой скоростям [1, 5, 8].

Таким образом, постановка задачи управления включает три взаимосвязанных этапа. Во-первых, необходимо преобразовать входные команды верхнего уровня, задаваемые в виде $v_{\text{зад}}(t)$ и $\omega_{\text{зад}}(t)$, в опорные параметры движения ведущих колёс. Во-вторых, необходимо определять фактические линейную и угловую скорости платформы по данным энкодеров ведущих колёс. В-третьих,

необходимо формировать такие управляющие воздействия на приводы, которые обеспечивают уменьшение ошибок $e_v(t)$ и $e_\omega(t)$ и, как следствие, воспроизведение требуемого режима движения платформы [1, 5, 11].

Полученная постановка задачи непосредственно подводит к синтезу закона управления, сочетающего использование математической модели и принципа обратной связи [1].

2.3. Синтез закона управления мобильной платформой

Для решения сформулированной задачи в работе используется комбинированный закон управления, включающий две составляющие: модельную упреждающую составляющую и корректирующую составляющую, формируемую по обратной связи. Применение такой структуры обусловлено тем, что математическая модель позволяет определить номинальный режим работы системы, соответствующий заданному движению платформы, однако не учитывает полностью влияние неидеальности приводов, внешних возмущений, разброса параметров и ошибок измерения. Введение корректирующего контура позволяет компенсировать возникающие отклонения и повысить точность воспроизведения требуемого режима движения [1, 6].

На первом этапе по заданным линейной и угловой скоростям платформы определяются требуемые линейные скорости ведущих колёс на основе кинематической модели дифференциальной платформы [5, 8, 11]:

$$v_{L,зад}(t) = v_{зад}(t) - \frac{L \omega_{зад}(t)}{2}$$
$$v_{R,зад}(t) = v_{зад}(t) + \frac{L \omega_{зад}(t)}{2}$$

На втором этапе полученные значения преобразуются в требуемые угловые скорости вращения колёс [5, 8]:

$$\omega_{L,зад}(t) = \frac{v_{L,зад}(t)}{r}$$

$$\omega_{R,зад}(t) = \frac{v_{R,зад}(t)}{r}$$

Полученные значения $\omega_{L,зад}(t)$ и $\omega_{R,зад}(t)$ задают номинальный режим вращения ведущих колёс, необходимый для реализации требуемого движения платформы. Однако сами по себе эти величины ещё не являются конечным управляющим воздействием, поскольку исполнительные механизмы приводятся в действие напряжением либо эквивалентным ему управляющим сигналом [14].

Для вычисления номинального управляющего воздействия используется статическая модель привода в установившемся режиме. В соответствии с данной моделью требуемое напряжение на двигателе определяется как сумма постоянной составляющей, необходимой для преодоления статических потерь, и линейной составляющей, пропорциональной требуемой угловой скорости вращения. Для левого и правого приводов номинальные напряжения определяются выражениями [14]:

$$U_{L,ff}(t) = k_S \operatorname{sgn}(\omega_{L,зад}(t)) + k_V \omega_{L,зад}(t)$$

$$U_{R,ff}(t) = k_S \operatorname{sgn}(\omega_{R,зад}(t)) + k_V \omega_{R,зад}(t)$$

где k_S — коэффициент статической составляющей напряжения, а k_V — коэффициент пропорциональности между требуемой угловой скоростью и напряжением двигателя в установившемся режиме.

Слагаемые $k_S \operatorname{sgn}(\omega_{L,зад}(t))$ и $k_S \operatorname{sgn}(\omega_{R,зад}(t))$ учитывают необходимость преодоления трения, зоны нечувствительности и других статических потерь в приводах. Слагаемые $k_V \omega_{L,зад}(t)$ и $k_V \omega_{R,зад}(t)$ отражают приближённую линейную зависимость между требуемой угловой скоростью вращения соответствующего колеса и напряжением двигателя в установившемся режиме. Следовательно, величины $U_{L,ff}(t)$ и $U_{R,ff}(t)$ представляют собой номинальные, или упреждающие, управляющие воздействия, рассчитываемые по модели без учёта текущего отклонения фактического движения платформы от заданного [14].

Однако в реальной системе использование только модельной составляющей, как правило, не обеспечивает требуемой точности

регулирования. Это связано с тем, что реальный объект не совпадает полностью с принятой моделью, а также подвержен действию внешних и внутренних возмущений. Поэтому в закон управления вводится корректирующая составляющая, формируемая по ошибкам линейной и угловой скоростей платформы [1, 6]:

$$e_v(t) = v_{зад}(t) - v(t)$$

$$e_\omega(t) = \omega_{зад}(t) - \omega(t)$$

В качестве корректирующего звена используются два ПИД-регулятора. Первый регулятор работает по ошибке линейной скорости платформы, второй — по ошибке её угловой скорости. Такой выбор обусловлен тем, что управление в рассматриваемой системе ориентировано на воспроизведение режима движения платформы как единого объекта, а не на независимое регулирование каждого колеса в отдельности [1, 6, 11].

Корректирующие воздействия определяются выражениями [6]:

$$u_v(t) = K_{Pv} e_v(t) + K_{Iv} \int_0^t e_v(\tau) d\tau + K_{Dv} \frac{de_v(t)}{dt}$$

$$u_\omega(t) = K_{P\omega} e_\omega(t) + K_{I\omega} \int_0^t e_\omega(\tau) d\tau + K_{D\omega} \frac{de_\omega(t)}{dt}$$

где $K_{P,v}, K_{I,v}, K_{D,v}$ — коэффициенты ПИД-регулятора линейной скорости, а $K_{P,\omega}, K_{I,\omega}, K_{D,\omega}$ — коэффициенты ПИД-регулятора угловой скорости.

Пропорциональная составляющая обеспечивает реакцию на текущее значение ошибки и способствует её оперативному уменьшению. Интегральная составляющая учитывает накопленную во времени ошибку и позволяет снижать установившееся рассогласование между заданным и фактическим режимом движения. Дифференциальная составляющая реагирует на скорость изменения ошибки и при корректной настройке улучшает характер переходного процесса и уменьшает колебательность системы [6].

Поскольку исполнительными механизмами платформы являются левый и правый приводы, корректирующие воздействия по линейной и угловой скорости должны быть преобразованы в добавочные управляющие воздействия для каждого канала. Для этого используется следующее смещение:

$$\Delta U_L(t) = u_v(t) - u_\omega(t)$$

$$\Delta U_R(t) = u_v(t) + u_\omega(t)$$

Итоговые управляющие воздействия на левый и правый приводы формируются как сумма модельной и корректирующей составляющих [1, 5, 6]:

$$U_L(t) = U_{L,ff}(t) + \Delta U_L(t)$$

$$U_R(t) = U_{R,ff}(t) + \Delta U_R(t)$$

Или, подставляя выражения для корректирующих воздействий, можно записать

$$U_L(t) = U_{L,ff}(t) + u_v(t) - u_\omega(t)$$

$$U_R(t) = U_{R,ff}(t) + u_v(t) + u_\omega(t)$$

Полученные величины $U_L(t)$ и $U_R(t)$ представляют собой конечный результат работы закона управления и подаются на исполнительные механизмы приводов. Таким образом, синтезированный закон управления реализует последовательную цепочку преобразований: от задания режима движения платформы в терминах линейной и угловой скоростей — к вычислению требуемых линейных, а затем угловых скоростей колёс, к определению номинальных напряжений на приводах и далее к их коррекции по сигналам обратной связи [1, 5, 6, 14].

Содержательно работа синтезированного закона управления может быть описана следующим образом. По входной команде движения сначала вычисляются требуемые линейные скорости ведущих колёс, после чего они преобразуются в требуемые угловые скорости их вращения. Затем по модели привода определяется напряжение, которое в идеализированном случае должно обеспечить данный режим. После этого на основе данных энкодеров ведущих колёс определяется отклонение фактических линейной и угловой скоростей платформы от требуемых значений, и ПИД-регуляторы формируют корректирующие воздействия по двум каналам управления. На заключительном

этапе эти корректирующие воздействия преобразуются в добавки к левому и правому приводам и суммируются с модельной составляющей. Тем самым система реализует замкнутый принцип управления, в котором модельная часть задаёт номинальное поведение, а обратная связь обеспечивает его уточнение в реальных условиях [1, 5, 6, 14].

Следовательно, синтезированный закон управления обеспечивает переход от абстрактной команды движения верхнего уровня к физически реализуемым управляющим воздействиям на приводы мобильной платформы. Использование кинематической модели позволяет определить требуемые параметры движения колёс, модель привода — вычислить номинальные напряжения, а ПИД-регуляторы по линейной и угловой скорости платформы — компенсировать отклонения, возникающие при работе реального объекта. В совокупности это формирует теоретическую основу программной реализации алгоритма управления, рассматриваемой в следующей главе [1, 5, 6, 8, 14].

В практической реализации синтезированного закона управления используются конкретные значения коэффициентов модели привода и коэффициентов ПИД-регуляторов. В рамках настоящей работы данные параметры определялись методом итерационной инженерной настройки в среде моделирования. Подбор выполнялся последовательным изменением коэффициентов с последующим запуском моделирования и анализом поведения замкнутой системы. В качестве основного критерия настройки использовалось уменьшение ошибки регулирования по линейной и угловой скорости при сохранении устойчивого характера переходного процесса и отсутствии нарастающих колебаний [6].

Итоговые значения коэффициентов, использованные в реализованной системе управления, приведены в таблице 1.

Таблица 1 — Значения коэффициентов модели привода и регуляторов

Параметр	Значение	Назначение
k_S	0,06	коэффициент статической составляющей модели привода
k_V	0,232	коэффициент скоростной составляющей модели привода
$K_{P,v}$	0,8	пропорциональный коэффициент регулятора линейной скорости
$K_{I,v}$	0,4	интегральный коэффициент регулятора линейной скорости
$K_{D,v}$	0,01	дифференциальный коэффициент регулятора линейной скорости
$K_{P,\omega}$	0,4	пропорциональный коэффициент регулятора угловой скорости
$K_{I,\omega}$	0,2	интегральный коэффициент регулятора угловой скорости
$K_{D,\omega}$	0,01	дифференциальный коэффициент регулятора угловой скорости

Испытания в среде моделирования проводились на программной реализации системы управления с использованием приведённых значений коэффициентов. Следовательно, результаты, представленные в последующих разделах работы, относятся именно к данному набору параметров системы управления.

2.4. Выводы

В результате построения математической модели и постановки задачи управления установлено, что управление мобильной платформой должно обеспечивать преобразование заданных линейной и угловой скоростей в номинальные параметры движения ведущих колёс с последующим формированием управляющих воздействий на левый и правый приводы. Тем самым задача управления сведена к обеспечению соответствия фактических линейной и угловой скоростей платформы их заданным значениям при использовании кинематической модели для вычисления номинального режима работы приводов [5, 8, 11].

В ходе синтеза закона управления показано, что рациональная структура системы включает модельную упреждающую составляющую и

корректирующую составляющую по обратной связи. Модельная часть позволяет по заданным параметрам движения определить требуемые угловые скорости колёс и соответствующие им номинальные напряжения приводов. Корректирующая часть, реализованная на основе ПИД-регуляторов по линейной и угловой скорости платформы, обеспечивает компенсацию отклонений, возникающих вследствие неидеальности реальной системы и действия возмущений [1, 6, 14].

Таким образом, в разделе получен закон управления, обеспечивающий переход от команды движения верхнего уровня к физически реализуемым управляющим воздействиям на приводы мобильной платформы.

3. Разработка программы

3.1. Назначение программы

Разрабатываемая программа предназначена для управления ходовой частью мобильного робота с дифференциальной схемой привода. Программа реализует вычислительный контур нижнего уровня и обеспечивает преобразование задаваемых параметров движения мобильной платформы в управляющие воздействия на левый и правый приводы ведущих колёс [5, 8, 11].

На вход программы поступает команда движения, задаваемая в виде требуемой линейной скорости платформы и требуемой угловой скорости платформы. Требуемая линейная скорость задаётся в метрах в секунду, а требуемая угловая скорость — в радианах в секунду. На основе этих величин программа вычисляет управляющие воздействия на приводы с учётом геометрических параметров платформы и данных обратной связи. Тем самым обеспечивается согласование между командой верхнего уровня, задаваемой в терминах движения платформы как единого объекта, и воздействиями, непосредственно подаваемыми на исполнительные механизмы [1, 5, 8, 11].

В процессе функционирования программа реализует комбинированный закон управления. Номинальная составляющая управляющего воздействия вычисляется на основе математической модели объекта управления и модели привода, а корректирующая составляющая формируется по данным обратной связи. В рамках текущей реализации источником обратной связи являются энкодеры ведущих колёс, по показаниям которых определяются фактические линейная и угловая скорости платформы, используемые в замкнутом контуре управления [1, 6, 14].

К входным сигналам программы относятся заданные значения линейной и угловой скоростей платформы, а также данные обратной связи, формируемые по измерениям энкодеров ведущих колёс. Геометрические параметры ходовой части и коэффициенты модели привода используются как настроечные параметры

вычислительного модуля. Выходом программы являются управляющие воздействия, формируемые отдельно для левого и правого приводов и передаваемые далее в платформенно-зависимую подсистему управления исполнительными механизмами [5, 6, 14].

С точки зрения общей структуры системы управления программа занимает промежуточное положение между уровнем задания движения и уровнем непосредственного управления приводами. Такое построение обеспечивает разделение вычислительной логики управления и платформенно-зависимых средств взаимодействия с аппаратурой, что упрощает перенос программного решения между моделирующей средой и целевой встраиваемой платформой [10, 12, 13].

Вычислительное ядро программы реализовано как платформенно-независимый программный модуль, содержащий алгоритмы формирования управляющих воздействий. Доступ к датчикам и исполнительным механизмам осуществляется через слой аппаратной абстракции, предоставляющий унифицированные программные интерфейсы. Это позволяет использовать один и тот же алгоритмический модуль в различных средах исполнения при замене только аппаратно-зависимой части системы [10, 12].

Таким образом, назначение разрабатываемой программы состоит в реализации замкнутого вычислительного цикла управления ходовой частью мобильного робота, включающего приём команды движения, обработку данных обратной связи, вычисление результирующих управляющих воздействий и их передачу в подсистему управления приводами. Программа обеспечивает программную реализацию выбранного закона управления и служит основой для воспроизведения требуемого режима движения мобильной платформы [1, 5, 6, 11].

3.2. Функциональные требования к программе

Функциональные требования к разрабатываемой программе определяются её назначением, принятой структурой алгоритма управления и

особенностями объекта управления, в качестве которого рассматривается мобильный робот с дифференциальной схемой привода. Программа должна обеспечивать выполнение вычислений, необходимых для преобразования команды движения верхнего уровня в управляющие воздействия на приводы ходовой части, а также использование обратной связи для коррекции результирующего управления [1, 5, 8, 11].

Программа должна обеспечивать приём команды движения, задаваемой в виде требуемой линейной скорости платформы, выраженной в метрах в секунду, и требуемой угловой скорости платформы, выраженной в радианах в секунду. Указанные величины должны использоваться в качестве основных входных сигналов вычислительного контура управления и служить исходными данными для расчёта управляющих воздействий на левый и правый приводы [5, 8, 11].

Программа должна обеспечивать преобразование заданных параметров движения платформы в управляющие воздействия на левый и правый приводы. При выполнении данного преобразования должны использоваться кинематическая модель дифференциальной платформы, геометрические параметры ходовой части, прежде всего радиус ведущих колёс и расстояние между ними, а также модель привода. Внутри вычислительного цикла при этом используется промежуточный расчёт параметров движения ведущих колёс, необходимых для формирования номинального управляющего воздействия [5, 8, 11, 14].

Программа должна обеспечивать вычисление номинального управляющего воздействия на основе используемой модели привода. Для этого должны применяться параметры, связывающие требуемую угловую скорость вращения колеса с управляющим воздействием на двигатель. Номинальное управление должно формироваться отдельно для левого и правого приводов в соответствии с расчётным режимом движения платформы [5, 14].

Программа должна обеспечивать получение и использование данных обратной связи о текущем состоянии ходовой части. В рамках реализуемого

решения такими данными являются значения, получаемые от энкодеров ведущих колёс. На их основе должны определяться фактические линейная и угловая скорости платформы, использоваться при вычислении ошибки регулирования и участвовать в формировании корректирующей составляющей управления [1, 5, 6].

Существенным функциональным требованием является реализация замкнутого принципа управления. Программа должна обеспечивать вычисление рассогласования между заданными и фактическими параметрами движения платформы и использовать полученное рассогласование при формировании результирующего управляющего воздействия. Это необходимо для уменьшения отклонений, обусловленных неточностью модели, различием характеристик приводов и действием внешних возмущений [1, 6, 14].

Программа должна обеспечивать совместное использование модельной и корректирующей составляющих управления. В ходе каждого цикла работы должно выполняться вычисление номинального управляющего воздействия, соответствующего заданному режиму движения, а также вычисление корректирующего воздействия по ошибкам линейной и угловой скоростей платформы. Итоговое управление должно формироваться отдельно для левого и правого приводов [1, 5, 6, 14].

К числу функциональных требований относится обеспечение дискретного циклического режима работы программы. Алгоритм управления должен выполняться с последовательным повторением операций приёма команды движения, считывания данных обратной связи, определения ошибки регулирования, вычисления номинальной и корректирующей составляющих управления и формирования выходных управляющих воздействий. Такая организация соответствует структуре цифровой системы управления и обеспечивает регулярное уточнение управления в процессе движения робота [7].

Программа должна обеспечивать формирование выходных управляющих воздействий, предназначенных для передачи в подсистему управления

исполнительными механизмами. Эти воздействия должны вырабатываться отдельно для левого и правого приводов и представляться в форме, пригодной для последующей передачи через слой аппаратной абстракции в платформенно-зависимую часть системы [10, 12].

Отдельным функциональным требованием является обеспечение режима остановки. При нулевых значениях требуемой линейной и угловой скоростей программа должна формировать управляющие воздействия, соответствующие прекращению движения платформы и переводу ходовой части в состояние покоя.

Программа должна обеспечивать отделение вычислительной логики управления от платформенно-зависимых средств взаимодействия с датчиками и исполнительными механизмами. Реализация алгоритмов вычисления управляющих воздействий не должна зависеть от конкретного способа доступа к аппаратным ресурсам. Взаимодействие с аппаратной частью должно осуществляться через слой аппаратной абстракции, предоставляющий унифицированные программные интерфейсы [10, 12].

Кроме того, программа должна обеспечивать возможность использования одного и того же вычислительного ядра как в моделирующей среде, так и при переносе на целевую встраиваемую платформу при наличии соответствующей аппаратно-зависимой реализации интерфейсов. Это позволяет рассматривать разрабатываемую программу как самостоятельный модуль нижнего уровня в составе более общей системы управления мобильным роботом [10, 12, 13].

Таким образом, функциональные требования к разрабатываемой программе включают приём команды движения, преобразование заданных параметров движения платформы в управляющие воздействия на приводы, использование энкодерной обратной связи, вычисление ошибки регулирования, формирование номинальных и корректирующих управляющих воздействий, раздельное управление левым и правым приводами, обеспечение дискретного циклического режима работы, поддержку режима остановки и аппаратную независимость вычислительного ядра программы.

3.3. Архитектура программы

Архитектура разработанной программы определяет состав её основных компонентов, их ответственность и порядок взаимодействия при реализации цикла управления мобильной платформой. Для рассматриваемой задачи архитектурное построение имеет принципиальное значение, поскольку программа должна объединять получение измерений, формирование текущего состояния платформы, вычисление управляющего воздействия и передачу команд исполнительным механизмам в составе единой согласованной системы [10, 12].

При построении архитектуры исходным требованием являлось разделение компонентов по их функциональной роли. Организация шага управления, формирование текущего состояния робота, вычисление закона управления, взаимодействие с датчиками и приводами, а также учёт особенностей конкретной среды исполнения относятся к различным задачам и потому не должны сосредотачиваться в одном программном модуле. Такое разделение позволяет уменьшить связанность компонентов, сделать границы их ответственности однозначными и обеспечить переносимость вычислительного ядра между различными средами исполнения [10, 12].

С учётом этого в составе программы выделены следующие основные архитектурные части: подсистема управляющей логики, вычислительная подсистема управления, подсистема аппаратной абстракции и платформенно-зависимая подсистема. Подсистема управляющей логики координирует выполнение шага управления и связывает между собой состояние робота, команду движения и вычислительные модули. Вычислительная подсистема формирует управляющее воздействие на основе целевой команды и текущего состояния платформы. Подсистема аппаратной абстракции задаёт унифицированный способ взаимодействия с

датчиками и исполнительными механизмами. Платформенно-зависимая подсистема содержит конкретные реализации этого взаимодействия для используемой среды исполнения [10, 12].

В архитектуре программы ключевую роль играют два компонента верхнего уровня: `RobotController` и `Robot`. Первый отвечает за вычисление управляющего воздействия, второй — за представление состояния платформы и взаимодействие с измерительной и исполнительной частью системы. Такое разделение является центральным архитектурным решением: вычисление управления и обслуживание аппаратного окружения не смешиваются в пределах одного класса, а взаимодействуют через явно определённые программные объекты [10, 12].

Общая структура архитектуры программы представлена на рисунке 2.

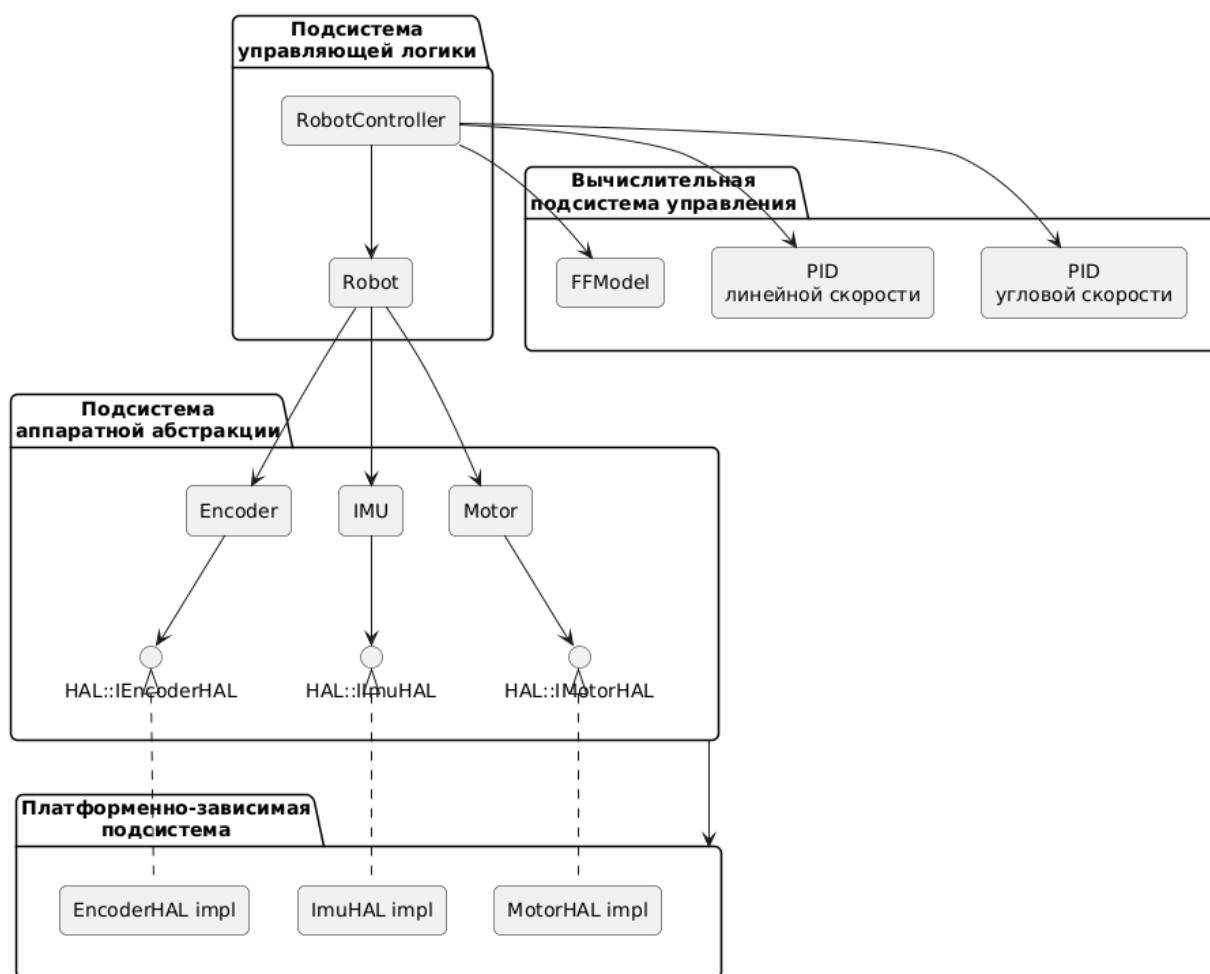


Рисунок 2 — Общая структура архитектуры программы

3.3.1. Подсистема управляющей логики

Подсистема управляющей логики образует верхний уровень архитектуры программы и определяет порядок выполнения полного шага управления мобильной платформой. На данном уровне не реализуется непосредственный доступ к платформенным средствам ввода-вывода, а вычисление закона управления вынесено в специализированные вычислительные модули. Назначение подсистемы состоит в координации взаимодействия между состоянием робота, целевой командой движения и результатом вычисления управляющего воздействия [10, 12].

Ключевым компонентом подсистемы является класс `RobotController`. Он получает доступ к объекту `Robot`, объекту модельного вычисления `FFModel`, а также к двум регуляторам `PID`, используемым для линейного и углового контуров управления. Тем самым `RobotController` выполняет роль координатора вычисления управляющего воздействия в пределах одного шага управления. Его основная задача состоит в том, чтобы на основе текущего состояния платформы и команды движения сформировать результирующее управляющее воздействие в виде структуры `ControlEffort`.

Вторым важнейшим компонентом подсистемы является класс `Robot`. Он представляет программную модель мобильной платформы и служит единой точкой взаимодействия с измерительной и исполнительной частью системы. Через него выполняются обновление датчиков, формирование текущего состояния робота и передача рассчитанных управляющих воздействий на исполнительный уровень. Внутри объекта `Robot` агрегируются средства работы с энкодерами, инерциальным датчиком, левым и правым мотором, а также компоненты, связанные с оценкой состояния платформы и сопровождением её движения.

С архитектурной точки зрения такое распределение ролей является принципиальным. RobotController отвечает за принятие решения о требуемом управляющем воздействии, тогда как Robot отвечает за подготовку входных данных для этого решения и за применение результата на нижнем уровне системы. Благодаря этому алгоритм управления, обработка измерений и работа с исполнительными механизмами остаются структурно разделёнными [10, 12].

Состав подсистемы управляющей логики приведён на рисунке 3.

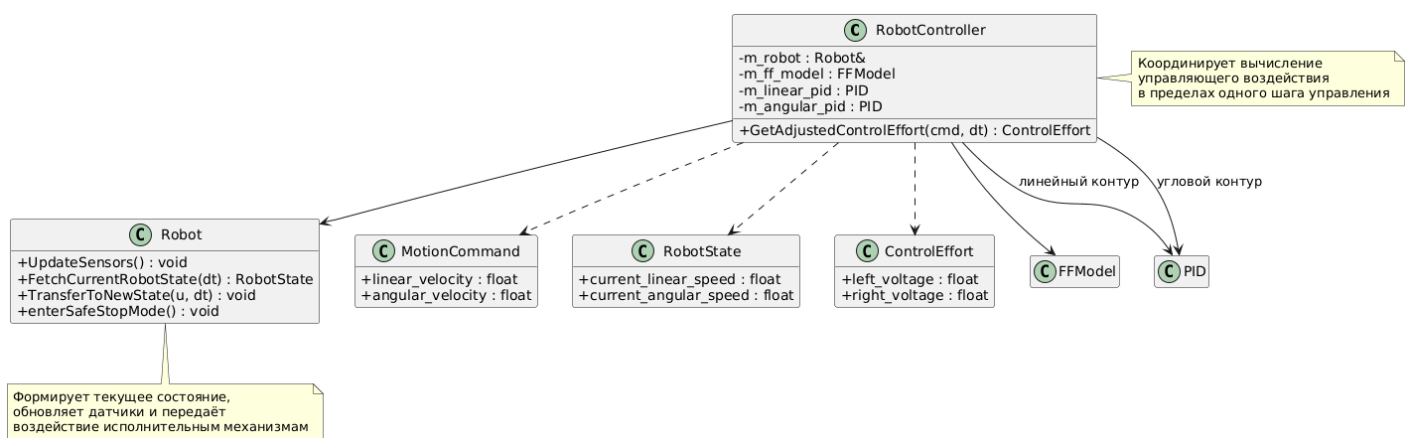


Рисунок 3 — Состав подсистемы управляющей логики

3.3.2. Вычислительная подсистема управления

Вычислительная подсистема управления представляет алгоритмическое ядро программы. Её назначение состоит в формировании управляющего воздействия на левый и правый приводы по заданной команде движения и текущему состоянию платформы. На уровне архитектуры данная подсистема не включает в себя процедур обновления датчиков, работы с аппаратными интерфейсами и непосредственной передачи команд исполнительным механизмам [10, 12].

В текущей реализации вычислительная подсистема построена как композиция модельной и корректирующей частей. Модельная часть представлена объектом FFModel, который преобразует команду движения

MotionCommand, содержащую требуемые линейную и угловую скорости платформы, в номинальное управляющее воздействие на левый и правый моторы. Для этого используются кинематические соотношения дифференциальной платформы и параметры модели привода, связывающие требуемую угловую скорость колеса с напряжением двигателя [5, 8, 11, 14].

Корректирующая часть реализована двумя регуляторами PID. Первый регулятор работает по ошибке линейной скорости платформы, второй — по ошибке её угловой скорости. Тем самым корректирующее воздействие формируется не по отдельным колёсам, а по параметрам движения платформы как единого объекта, что соответствует принятой постановке задачи управления [1, 6].

Итоговое управляющее воздействие формируется как сумма модельной и корректирующей составляющих. Таким образом, вычислительная подсистема реализует комбинированный закон управления: модельная часть задаёт номинальный режим движения, а корректирующая часть уменьшает отклонение фактических линейной и угловой скоростей платформы от требуемых значений [1, 6, 14].

Структура вычислительной подсистемы управления представлена на рисунке 4.

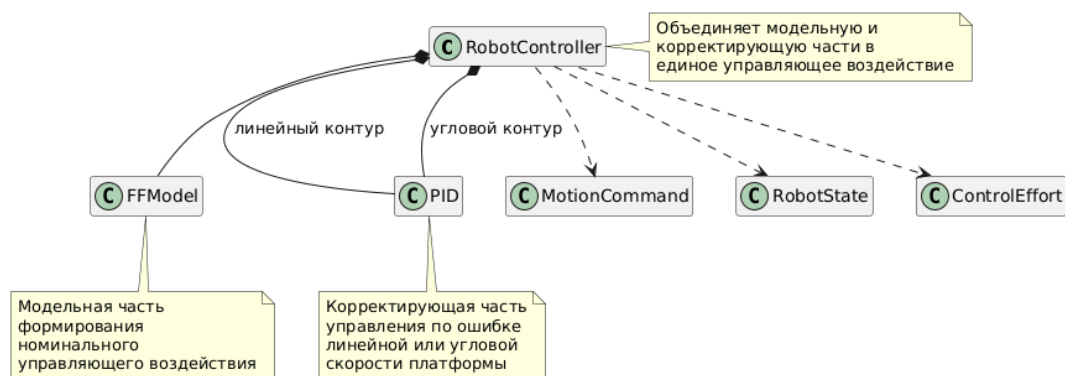


Рисунок 4 — Структуры вычислительной подсистемы управления

3.3.3. Подсистема аппаратной абстракции

Подсистема аппаратной абстракции предназначена для отделения вычислительной логики управления от конкретных средств взаимодействия с датчиками и исполнительными механизмами. На данном уровне определяется набор интерфейсов, через которые верхние компоненты программы получают измерения и передают команды управления, не завися при этом от конкретной платформенной реализации [10, 12].

В рассматриваемой программе подсистема аппаратной абстракции имеет двухуровневое строение. Первый уровень образуют интерфейсы `HAL::IEncoderHAL`, `HAL::IImuHAL` и `HAL::IMotorHAL`, задающие минимальный контракт взаимодействия с соответствующими устройствами. Эти интерфейсы определяют допустимые операции чтения и записи, но не содержат сведений о конкретной платформе или среде исполнения [10, 12].

Второй уровень образуют предметные обёртки `Encoder`, `IMU` и `Motor`, работающие поверх HAL-интерфейсов. Они предоставляют верхним компонентам программы более удобную и предметно-ориентированную форму доступа к устройствам. Благодаря этому класс `Robot` взаимодействует не с низкоуровневыми платформенными объектами напрямую, а с объектами предметной области, отражающими структуру робототехнической системы [10, 12].

Такое построение позволяет локализовать детали аппаратного взаимодействия на нижнем уровне и не распространять их на вычислительные модули и управляющую логику. Подсистема аппаратной абстракции тем самым образует промежуточный слой между алгоритмами управления и конкретной средой исполнения [10, 12].

Двухуровневая организация подсистемы аппаратной абстракции показана на рисунке 5.

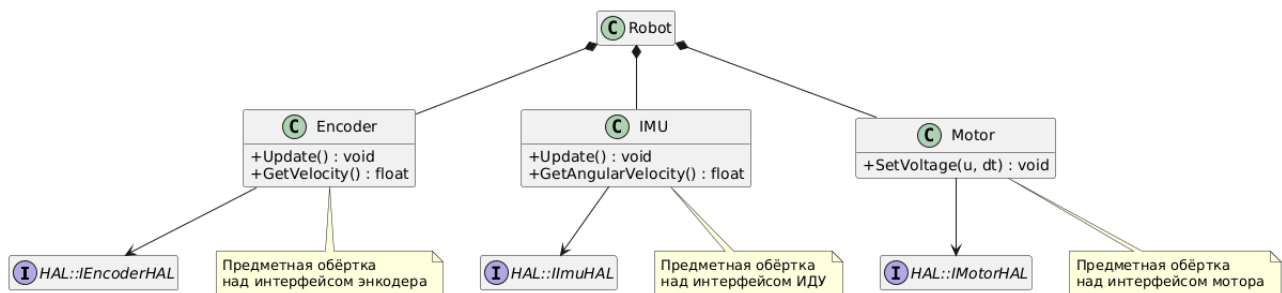


Рисунок 5 — Организация подсистемы аппаратной абстракции

3.3.4. Платформенно-зависимая подсистема

Платформенно-зависимая подсистема содержит конкретные реализации интерфейсов, определённых уровнем аппаратной абстракции. Если HAL задаёт, какие операции должны быть доступны верхним компонентам программы, то платформенно-зависимая подсистема определяет, каким образом эти операции выполняются в конкретной среде исполнения [10, 12].

На данном уровне сосредоточены классы, непосредственно связанные с выбранной платформой или моделирующей средой. Они реализуют получение данных от датчиков и передачу управляющих сигналов исполнительным механизмам в соответствии с особенностями конкретного окружения. При этом верхние компоненты программы не обращаются к данным реализациям напрямую, а используют только HAL-интерфейсы и предметные обёртки устройств [10, 12].

Такое построение позволяет локализовать платформенные особенности в пределах отдельного архитектурного уровня. В результате верхние уровни программы сохраняют единый состав и структуру

независимо от того, используется ли моделирующая среда или целевая встраиваемая платформа [10, 12, 13].

Состав платформенно-зависимой подсистемы представлен на рисунке 6.



Рисунок 6 — Состав платформенно-зависимой подсистемы

3.3.5. Взаимодействие подсистем в цикле управления

Для полноты архитектурного описания необходимо рассмотреть порядок взаимодействия выделенных подсистем в пределах одного шага управления. Именно этот порядок связывает отдельные компоненты в единую программную систему и задаёт реальную структуру выполнения цикла управления [10, 12].

На каждом шаге работы компонент Robot сначала обновляет данные датчиков. После этого он формирует текущее состояние платформы, включающее фактические линейную и угловую скорости, вычисляемые по данным энкодеров. Подготовленное состояние используется компонентом RobotController совместно с командой движения для вычисления результирующего управляющего воздействия. Далее рассчитанное воздействие передаётся обратно в Robot, который обеспечивает его применение к исполнительному уровню через моторные обёртки и нижележащие интерфейсы аппаратной абстракции.

Тем самым в пределах одного шага управления класс Robot связывает вычислительный уровень с измерительной и исполнительной частью системы, тогда как RobotController выполняет координацию вычисления управляющего воздействия. Вычислительные модули FFModel и PID используются внутри этого процесса как специализированные компоненты

формирования номинальной и корректирующей частей управления. Подсистема аппаратной абстракции обеспечивает единый способ работы с устройствами, а платформенно-зависимая подсистема реализует этот способ применительно к конкретной среде исполнения [1, 6, 10, 12].

Важной особенностью такой схемы является то, что вычислительная часть не взаимодействует с платформенными реализациями напрямую. Она получает уже подготовленное состояние платформы и возвращает результат вычисления в формализованном виде. Это позволяет сохранить чёткие границы ответственности между получением измерений, вычислением управления и передачей сигналов на исполнительные механизмы [10, 12].

3.3.6. Выводы

Архитектура разработанной программы построена на разделении системы на подсистему управляющей логики, вычислительную подсистему управления, подсистему аппаратной абстракции и платформенно-зависимую подсистему. Ключевую роль в данной архитектуре играют компоненты RobotController и Robot, вычислительные модули FFModel и PID, HAL-интерфейсы и предметные обёртки устройств. Такое разбиение позволяет отделить организацию шага управления от вычисления управляющего воздействия и от конкретных механизмов взаимодействия с устройствами.

Тем самым архитектура программы задаёт согласованную структуру компонентов, их границы ответственности и порядок взаимодействия в составе общего цикла управления мобильной платформой. Верхние уровни программы работают с формализованным состоянием платформы и управляющим воздействием, тогда как особенности конкретной среды исполнения локализованы на нижнем платформенном уровне. Это обеспечивает целостность программной системы и создаёт основу для

последующей детализации алгоритмов, рассматриваемых в следующем разделе.

3.4. Программная реализация цикла управления

В данном разделе рассматривается программная реализация цикла управления ходовой частью мобильной платформы. В отличие от предыдущего раздела, где основное внимание уделялось составу компонентов и их архитектурным ролям, здесь рассматривается порядок работы этих компонентов в пределах одного шага управления. Изложение строится от общего шага управления к отдельным процедурам, обеспечивающим обновление измерений, формирование текущего состояния платформы, вычисление управляющего воздействия, его передачу исполнительным механизмам и выполнение вспомогательных операций, связанных с устойчивостью и безопасностью функционирования системы [10, 12].

Разрабатываемая система управления реализует дискретный замкнутый контур. Это означает, что управление формируется не непрерывно, а на отдельных шагах дискретизации, в каждом из которых последовательно выполняются обновление данных датчиков, формирование текущего состояния платформы, вычисление нового управляющего воздействия и его передача на исполнительные механизмы [1, 7]. Такая последовательность действий распределена между компонентами Robot, RobotController, FFModel, PID, а также предметными обёртками Encoder, IMU и Motor, работающими поверх интерфейсов аппаратной абстракции [10, 12].

С точки зрения решаемой задачи программный контур ориентирован на управление скоростным режимом движения платформы. На вход вычислительной подсистемы подаётся команда движения MotionCommand, содержащая требуемые линейную и угловую скорости, а на выходе формируется структура ControlEffort, задающая управляющие напряжения левого и правого моторов. Следовательно, реализованный контур предназначен для стабилизации

линейной и угловой скоростей мобильной платформы, а не для непосредственного управления её координатами в пространстве [1, 5, 8, 11].

С инженерной точки зрения такая организация обеспечивает детерминированный порядок работы и согласуется с архитектурой программы. Компонент Robot отвечает за обновление датчиков, формирование текущего состояния и передачу напряжений на моторы, тогда как RobotController реализует вычисление управляющего воздействия. Благодаря этому обработка измерений, вычисление управления и применение сигнала к исполнительным механизмам разделены по своим функциональным ролям [10, 12]. Подробная блок-схема программной реализации общего шага управления приведена в приложении А.

3.4.1. Общий шаг управления

В программной реализации один шаг управления представляет собой последовательность трёх основных стадий. На первой стадии выполняется обновление состояний датчиков. На второй стадии по текущему состоянию платформы и требуемой команде движения вычисляется управляющее воздействие. На третьей стадии рассчитанное воздействие передаётся исполнительным механизмам. Именно такой порядок работы закреплён в структуре программного цикла и определяет общий алгоритм функционирования системы [7, 10, 12].

Формально один шаг управления может быть представлен отображением

$$u_k = F(x_k, u_k^{zad}, \Delta t)$$

где x_k — текущее состояние платформы, u_k^{zad} — команда движения, содержащая требуемые линейную и угловую скорости, Δt — шаг дискретизации, а u_k — вычисленное управляющее воздействие на левый и правый приводы.

В текущей реализации значение Δt передаётся в алгоритмы как внешний параметр. Это позволяет учитывать фактический интервал времени между соседними вызовами цикла и одновременно выполнять проверку его

допустимости. Таким образом, временная дисциплина является встроенным элементом надёжности программной реализации [7].

Алгоритм одного шага управления может быть представлен следующим образом.

Алгоритм 3.1 — Общий шаг управления

Вход: команда движения $u_k^{зад}$, шаг дискретизации Δt

1. обновить состояния датчиков;
2. сформировать текущее состояние платформы x_k ;
3. вычислить управляющее воздействие u_k ;
4. передать u_k исполнительным механизмам.

Таким образом, общий шаг управления задаёт координирующую основу функционирования всей системы. Содержимое отдельных стадий раскрывается частными алгоритмами, рассматриваемыми далее.

Последовательность выполнения одного шага управления показана на рисунке 7.

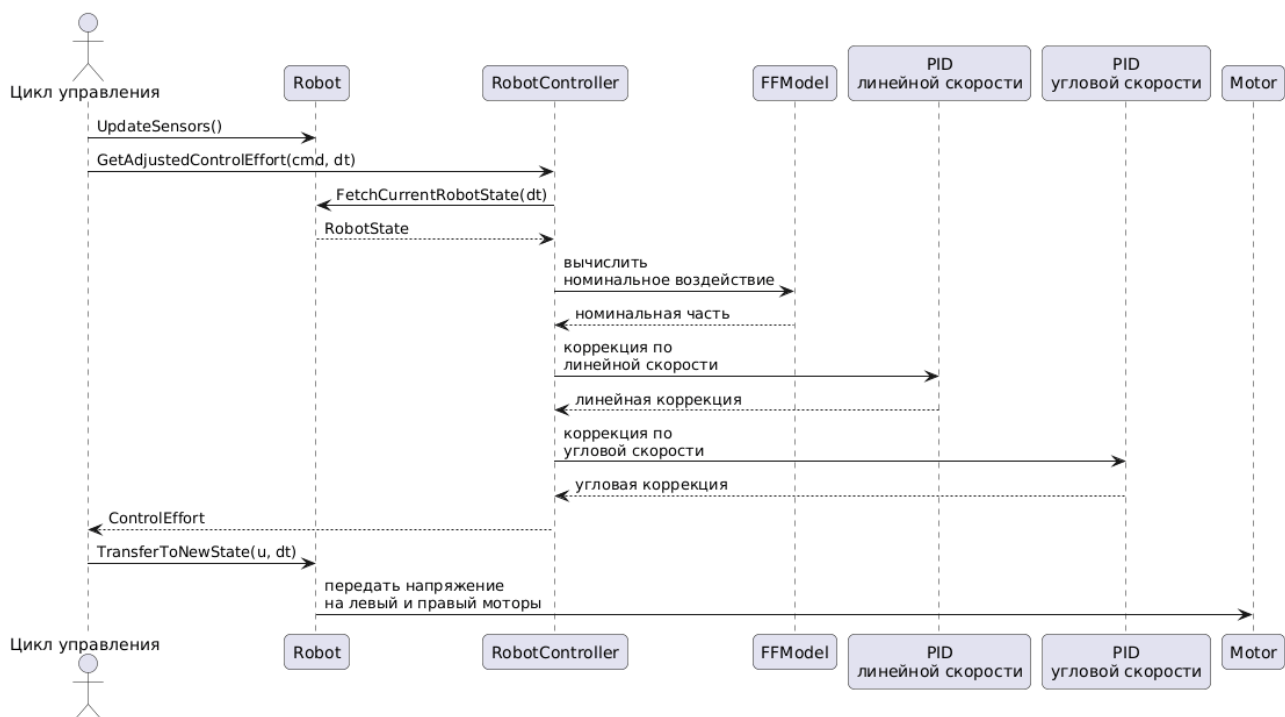


Рисунок 7 — Последовательность выполнения одного шага управления

3.4.2. Обновление датчиков и формирование текущего состояния платформы

Формирование текущего состояния платформы в рассматриваемой реализации разделено на два этапа. Сначала компонент Robot вызывает процедуру UpdateSensors(), инициирующую обновление внутренних состояний энкодеров и инерциального датчика. Затем вызывается процедура FetchCurrentRobotState(dt), в рамках которой выполняются проверка допустимости шага дискретизации, валидация энкодеров, вычисление параметров движения платформы и обновление вспомогательных объектов сопровождения состояния.

Для энкодеров в коде реализована фильтрация измеренной линейной скорости с использованием экспоненциального сглаживания вида

$$v_k^f = (1 - \alpha) v_{k-1}^f + \alpha v_k^{\text{сырая}}$$

В текущей реализации коэффициент α принят равным 1,0, поэтому фильтр фактически пропускает текущее измерение без дополнительного сглаживания. Тем не менее сама процедура фильтрации уже выделена структурно и может быть перенастроена без изменения остальной части контура управления.

После обновления датчиков компонент Robot вычисляет текущие линейную и угловую скорости платформы по скоростям левого и правого колёс. Для дифференциальной схемы привода используются соотношения [5, 8, 11]:

$$v = \frac{v_L + v_R}{2}$$

$$\omega = \frac{v_R - v_L}{B}$$

где B — расстояние между ведущими колёсами. Полученные значения записываются в структуру `RobotState` как текущие линейная и угловая скорости платформы. Именно эти величины в текущей реализации передаются далее в вычислительную подсистему управления.

В состав компонента `Robot` также включён объект `RobotEKF`, используемый для оценки состояния по данным энкодеров и инерциального датчика. Внутри `FetchCurrentRobotState(dt)` выполняются шаги прогноза и коррекции этой оценки, однако итоговые значения скоростей, используемые контуром управления, в текущей версии берутся непосредственно из энкодерно-кинематического расчёта. Следовательно, фильтр Калмана следует рассматривать как уже встроенную, но пока не подключённую в основной контур инфраструктуру оценки состояния.

Дополнительно на данном этапе обновляется объект одометрии. Расчёт положения выполняется по соотношениям [5, 8, 11]:

$$x_{k+1} = x_k + v_k \cos \theta_k \cdot \Delta t$$

$$y_{k+1} = y_k + v_k \sin \theta_k \cdot \Delta t$$

$$\theta_{k+1} = \theta_k + \omega_k \cdot \Delta t$$

В текущем контуре эти координатные оценки не используются непосредственно при вычислении управления, однако позволяют сопровождать движение платформы дополнительной информацией о её положении и ориентации.

Таким образом, результатом данного этапа является согласованная структура текущего состояния робота, пригодная для последующего вычисления управляющего воздействия. Схема формирования текущего состояния платформы приведена на рисунке 8.

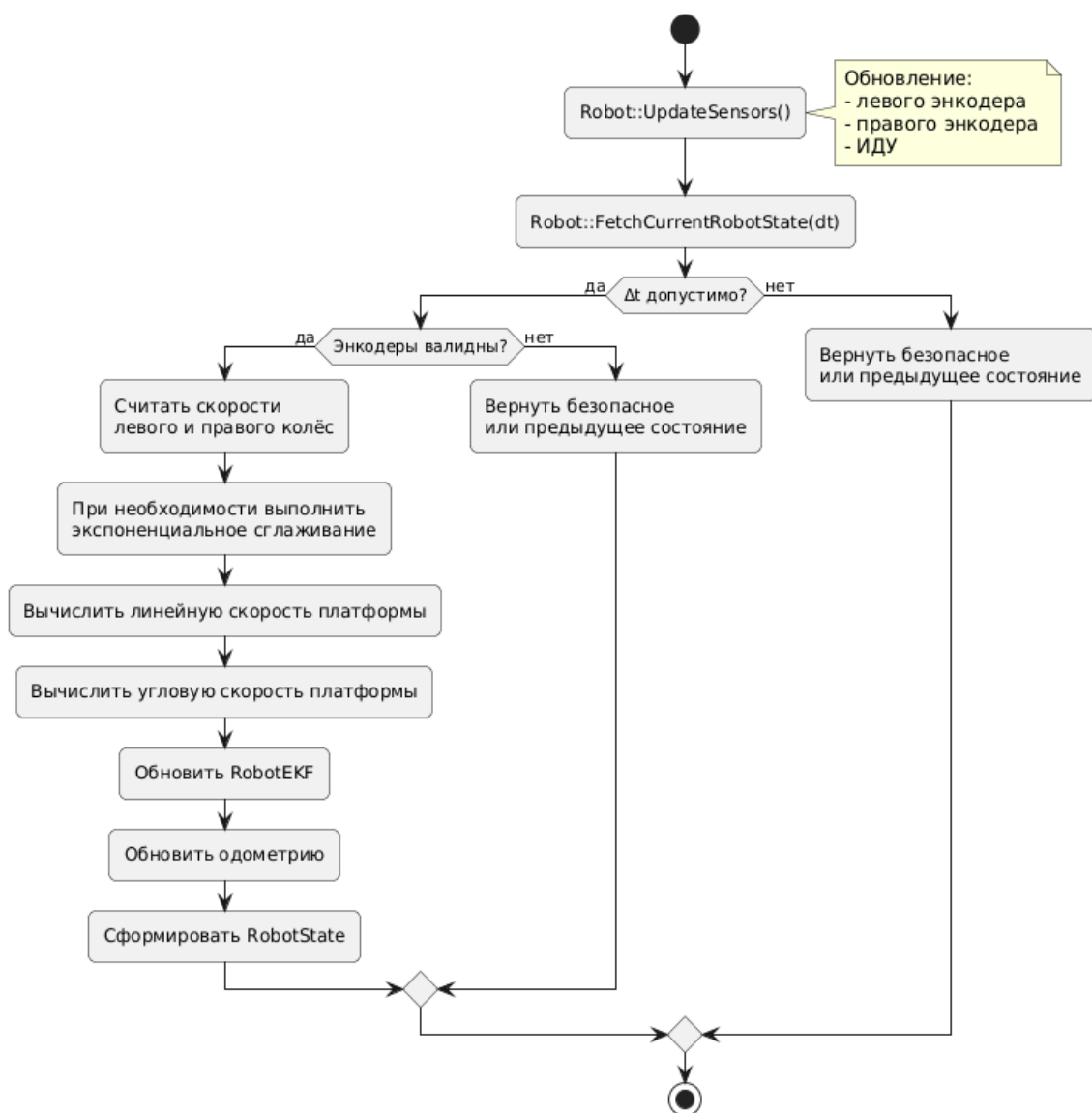


Рисунок 8 — Схема формирования текущего состояния платформы

3.4.3. Вычисление управляющего воздействия

Центральным алгоритмом программной реализации является вычисление управляющего воздействия, выполняемое методом `RobotController::GetAdjustedControlEffort(...)`. На вход алгоритма поступают команда движения `MotionCommand`, содержащая требуемые линейную и угловую скорости, и шаг дискретизации Δt . На выходе формируется структура `ControlEffort`, содержащая напряжения левого и правого моторов.

На первом этапе выполняется проверка корректности шага дискретизации и текущего безопасного состояния платформы. При слишком большом Δt либо при уже активированном безопасном режиме компонент `RobotController`

инициирует остановку, сбрасывает состояния регуляторов и возвращает нулевое воздействие. При слишком малом Δt возвращается последнее безопасное воздействие. Следовательно, в алгоритм вычисления управления встроены защитные механизмы, ограничивающие работу регулятора вне допустимых условий [7, 10, 12].

На следующем этапе производится сглаживание входной команды движения с помощью процедуры `VelocityRamp(...)`. Линейная и угловая команды не изменяются скачком, а постепенно приближаются к требуемым значениям с ограничением максимально допустимого изменения за один шаг:

$$v_k^{cgl} = v_{k-1}^{cgl} + \text{огр}\left(v_k^{\text{зад}} - v_{k-1}^{cgl}; a_{\text{лин}} \Delta t\right)$$

где a_v и a_ω задают максимально допустимую скорость изменения линейной и угловой команды соответственно. Такая процедура уменьшает резкость переходных процессов и улучшает согласование вычислительной подсистемы с исполнительными механизмами.

После сглаживания команды вычисляется номинальная модельная составляющая управляющего воздействия. Для этого используется объект `FFModel`, который сначала по кинематике дифференциального привода вычисляет линейные скорости левого и правого колёс, затем переводит их в угловые скорости, а после этого рассчитывает напряжения левого и правого моторов по статической модели привода [5, 8, 11, 14]:

$$U = k_s \text{sgn}(\omega) + k_v \omega$$

Тем самым `FFModel` реализует модельную часть закона управления, задающую номинальное воздействие, соответствующее требуемому режиму движения [1, 14].

Корректирующая часть управления реализована двумя ПИД-регуляторами: один работает по ошибке линейной скорости платформы, второй — по ошибке её угловой скорости. Это соответствует схеме регулирования по

параметрам движения платформы как единого объекта, а не по отдельным скоростям колёс [1, 6].

Полученные линейная и угловая корректирующие составляющие преобразуются в добавочные напряжения левого и правого каналов:

$$u_{L,k}^{кор} = u_{v,k} - u_{\omega,k}$$

$$u_{R,k}^{кор} = u_{v,k} + u_{\omega,k}$$

После этого к ним добавляется модельная составляющая:

$$u_{L,k} = u_{L,k}^{ff} + u_{L,k}^{cor}$$

$$u_{R,k} = u_{R,k}^{ff} + u_{R,k}^{cor}$$

Именно эта сумма образует полное требуемое управляющее воздействие на моторы.

Следовательно, реализованный закон управления имеет комбинированный характер. Математическая модель задаёт номинальный уровень воздействия, а ПИД-регуляторы компенсируют отклонение фактической линейной и угловой скорости платформы от требуемого значения [1, 6, 14]. Схема вычисления управляющего воздействия приведена на рисунке 9.

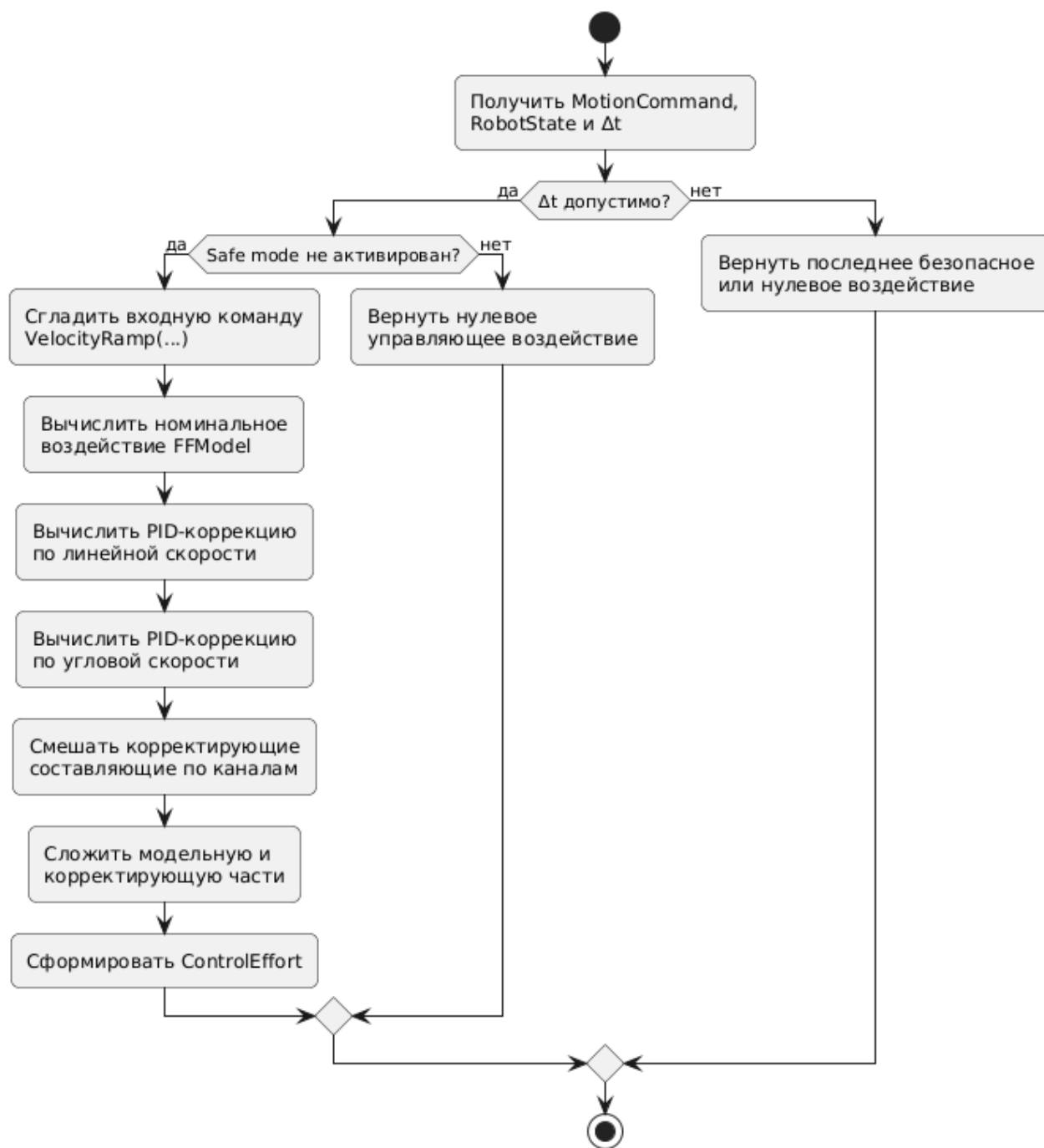


Рисунок 9 — Схема вычисления управляющего воздействия

3.4.4. Ограничение и передача управляющего воздействия исполнительным механизмам

После вычисления полного управляющего воздействия выполняется его согласование с исполнительным уровнем. На стороне RobotController для этого используется объект ActuatorLimits, которому передаётся рассчитанная структура ControlEffort. Архитектурно данный объект предназначен для

реализации физических ограничений исполнительных механизмов и формирования допустимого управляющего воздействия. В текущей версии реализации метод `ApplyLimits(...)` возвращает запрошенное воздействие без изменения и помечает его как ненасыщенное. Следовательно, сам слой ограничений уже выделен в архитектуре и встроен в контур управления, хотя его логика пока не доведена до полной функциональной реализации.

Если ограничитель сообщает о насыщении, контроллер выполняет процедуру компенсации насыщения интегральной составляющей регуляторов. Для этого вычисляется разность между желаемым и фактически допустимым напряжением на левом и правом каналах, после чего эта разность преобразуется обратно в линейную и угловую составляющие и передаётся в ПИД-регуляторы через механизм обратной коррекции интегральной части. Тем самым в программной структуре уже предусмотрен механизм предотвращения накопления интегральной ошибки в условиях насыщения исполнительных воздействий [6].

Фактическая передача сигнала на моторы выполняется методом `Robot::TransferToNewState(...)`. Внутри него повторно вызывается ограничитель, проверяется безопасный режим, работоспособность моторов и допустимость напряжений, после чего рассчитанные значения передаются левому и правому мотору. Если условия безопасной работы нарушаются, система переводится в безопасный режим. Таким образом, стадия передачи воздействия выполняет не только функцию вывода сигнала, но и функцию окончательной проверки допустимости его применения к исполнительным механизмам.

Класс `Motor` реализует дополнительное согласование вычисленного напряжения с особенностями привода. Перед передачей на нижний уровень значение ограничивается по максимально допустимому напряжению, после чего изменяется плавно с учётом коэффициента нарастания и величины шага дискретизации. Следовательно, выходное воздействие не подаётся на мотор

мгновенным скачком, а проходит через процедуру плавного изменения, уменьшающую резкость переходных процессов [14].

Схема передачи управляющего воздействия на исполнительные механизмы приведена в приложении Б.

3.4.5. Вспомогательные процедуры обеспечения устойчивости и безопасной работы

Помимо основного закона управления программная реализация включает набор вспомогательных процедур, влияющих на устойчивость и безопасный характер функционирования системы. К ним относятся проверка допустимости шага дискретизации, валидация работоспособности датчиков и моторов, фильтрация измерений, сглаживание команды движения и перевод системы в безопасный режим. Эти процедуры не образуют самостоятельного закона управления, однако существенно влияют на практическую применимость всей системы [7, 10, 12].

В коде реализованы явные ограничения на допустимый шаг дискретизации. Для компонента RobotController верхний предел задан как 0,5 с, а нижний — как 10⁻⁶ с. Для компонента Robot используется более жёсткое ограничение верхнего предела 0,05 с при том же нижнем пределе. Выход за допустимые границы приводит к возврату последнего безопасного результата либо к переводу системы в безопасный режим. Следовательно, временная корректность входных данных рассматривается в реализации как обязательное условие допустимой работы контура управления [7].

Безопасный режим реализован методом enterSafeStopMode(). После его активации устанавливается внутренний признак небезопасного состояния, а напряжение на левом и правом моторах поэтапно снижается до нуля, после чего на аппаратный уровень передаются нулевые значения напрямую. Одновременно вычислительная подсистема прекращает нормальное формирование управляющего воздействия и возвращает нулевое управление. Таким образом,

безопасный режим выполняет функцию аварийной остановки, встроенной в общий алгоритм работы системы.

Следует отметить, что часть защитной логики уже полностью участвует в работе системы, тогда как часть оформлена как задел для последующего развития. Это относится прежде всего к модулю ограничений исполнительных воздействий, который архитектурно встроен в контур, но пока не реализует реального насыщения. Поэтому в текущей версии корректно говорить не о полностью завершённой системе физических ограничений, а о частично реализованной инфраструктуре безопасного управления, в которой основные проверки уже присутствуют, а отдельные элементы ещё допускают дальнейшее развитие.

Схема вспомогательных процедур обеспечения устойчивости и безопасной работы показана в приложении В.

3.4.6. Связь реализованных алгоритмов с архитектурой системы

Рассмотренные алгоритмы непосредственно соответствуют функциональному разделению, принятому в архитектуре программы. Обновление датчиков, формирование текущего состояния, обновление одометрии и передача сигналов исполнительным механизмам сосредоточены в компоненте Robot. Вычисление модельной и корректирующей частей управления сосредоточено в RobotController, FFModel и объектах PID. HAL-интерфейсы и предметные обёртки Encoder, IMU и Motor обеспечивают связь между вычислительной логикой и конкретной реализацией аппаратного уровня [10, 12].

Благодаря такому распределению обязанностей программная реализация цикла управления сохраняет внутреннюю согласованность с архитектурным описанием системы. Верхний уровень не взаимодействует с датчиками и приводами напрямую, а опирается на компонент Robot и вычислительные модули. В свою очередь, формирование управляющего воздействия отделено от процедур получения измерений и от конкретного способа передачи напряжения

на моторы. Это обеспечивает логическую целостность программной системы и делает возможным её дальнейшее поэтапное развитие [10, 12].

3.4.7. Выводы

В программной реализации системы управления ходовой частью мобильной платформы реализован дискретный замкнутый цикл, включающий обновление датчиков, формирование текущего состояния платформы, вычисление управляющего воздействия и передачу вычисленного сигнала исполнительным механизмам. Управление строится как сумма модельной составляющей и корректирующей ПИД-составляющей по линейной и угловой скорости платформы, после чего результат проходит через слой ограничений и согласования с приводами.

Существенной особенностью текущей реализации является её практическая инженерная направленность. В контур уже встроены фильтрация измерений, сглаживание входной команды, проверка допустимости шага дискретизации и безопасный режим остановки. Вместе с тем некоторые элементы, в частности полноценная реализация насыщения через `ActuatorLimits` и использование оценки ЕКФ как основного источника скоростей для управления, пока выступают как задел для последующего развития. Поэтому описанный программный цикл следует рассматривать как работоспособную первую реализацию контура управления, обладающую целостной структурой и допускающую дальнейшее уточнение отдельных компонентов.

3.5. Тестирование

Тестирование разработанной программы выполняется с целью подтверждения корректности реализации отдельных компонентов системы управления и проверки их совместной работы в условиях, приближённых к реальному циклу функционирования мобильной платформы. Для рассматриваемой программной системы данная задача имеет принципиальное значение, поскольку управление движением робота реализуется как дискретный

замкнутый процесс, включающий получение данных от датчиков, формирование текущего состояния платформы, вычисление управляющего воздействия и его передачу исполнительным механизмам. Ошибка в любом из перечисленных этапов может привести к искажению поведения всей системы в целом [1, 7, 10, 12].

В соответствии с принятой архитектурой тестирование в работе разделено на два взаимодополняющих уровня. На первом уровне выполняется модульное тестирование, ориентированное на изолированную проверку отдельных программных компонентов. На втором уровне выполняется тестирование в среде моделирования, позволяющее проверить совместную работу реализованных модулей в составе единого контура управления. Такое разделение соответствует общей инженерной логике разработки: сначала подтверждается корректность базовых вычислительных и логических модулей, а затем исследуется поведение полной системы в интеграционном режиме [2, 10, 12].

Использование двухуровневой схемы тестирования позволяет решить сразу несколько задач. Во-первых, обеспечивается раннее выявление ошибок в отдельных алгоритмах до начала комплексных испытаний. Во-вторых, появляется возможность локализовать причину обнаруженного отклонения, поскольку на уровне модульных тестов каждый компонент проверяется в контролируемой среде. В-третьих, результаты модульных испытаний создают основу для последующего анализа интеграционного поведения программы, при котором уже оценивается не только корректность отдельных функций, но и согласованность их совместной работы [10, 12].

Таким образом, тестирование в рамках данной работы рассматривается не как завершающий формальный этап, а как неотъемлемая часть процесса разработки, обеспечивающая подтверждение корректности программной реализации и создающая основу для последующей оценки качества функционирования всей системы управления [10, 12].

3.5.1. Модульное тестирование

Модульное тестирование программного обеспечения системы управления ходовой частью мобильного робота выполняется с целью проверки корректности реализации отдельных программных компонентов в изолированной программной среде. Такой подход позволяет выявлять ошибки на уровне отдельных классов, вычислительных модулей и логических процедур до проведения интеграционных испытаний. Для рассматриваемой работы это имеет принципиальное значение, поскольку система управления построена как совокупность взаимосвязанных, но функционально разграниченных компонентов, корректность которых целесообразно подтверждать до проверки полного замкнутого контура [10, 12].

В текущей реализации модульное тестирование выполнено с использованием библиотеки GoogleTest. Тестовая конфигурация оформлена как отдельный исполняемый модуль `test_robot`, включающий набор тестовых файлов и имитационные реализации интерфейсов аппаратной абстракции. Это позволяет запускать тесты в `host`-среде без использования целевой аппаратной платформы и тем самым изолировать вычислительные и логические компоненты от аппаратно-зависимого окружения. В составе тестовой конфигурации применяются имитационные реализации HAL для энкодеров, приводов и инерциального датчика, что обеспечивает воспроизводимую подачу входных воздействий и контролируемый анализ поведения тестируемых модулей [10, 12, 19].

В соответствии с принятой архитектурой программы модульному тестированию подвергнуты прежде всего те компоненты, которые допускают автономную проверку: модуль оценки состояния `RobotEKF`, классы работы с датчиками и исполнительными механизмами (`Encoder`, `IMU`, `Motor`), одометрическая подсистема `Odometry`, вычислительные модули `FFModel` и `PID`, а также интегрирующие компоненты `Robot` и `RobotController`. Структура набора тестов соответствует структуре исходного кода: для каждого из указанных

компонентов выделен отдельный файл тестов, в котором проверяются характерные режимы работы и граничные случаи. По составу тестовых файлов и объявленных тестовых случаев набор включает 9 тестовых групп и 51 отдельное испытание. Сводные сведения о модульном тестировании приведены в таблице 1 [10, 12, 19].

Таблица 1 — Сводные результаты модульного тестирования

Компонент	Количество тестов	Результат	Проверяемые аспекты
RobotEKF	10	пройдены	инициализация, сброс состояния, шаг прогноза, коррекция по измерениям, работа при изменении шумовых параметров, корректность обновления состояния
Motor	6	пройдены	нарастание и спад управляющего воздействия, выход на требуемое значение, ограничение напряжения, поведение при нулевом шаге времени
Odometry	3	пройдены	прямолинейное движение, поворот, движение по замкнутой траектории
Math::PID	8	пройдены	реакция на изменение задания, ограничение выхода, компенсация насыщения, реакция на возмущение, поведение дифференциальной составляющей
Encoder	5	пройдены	фильтрация скорости, корректность чтения данных, обработка нештатных входных значений, восстановление после отказа
FFModel	6	пройдены	корректность вычисления номинального управляющего воздействия, ограничение напряжения, учёт статической и скоростной составляющих модели

Компонент	Количество тестов	Результат	Проверяемые аспекты
IMU	6	пройдены	фильтрация сигнала, корректность обработки измерений по осям, отсутствие побочных эффектов при чтении, восстановление после отказа
Robot	5	пройдены	формирование состояния робота, обработка нештатных ситуаций, применение ограничений безопасности, безопасная остановка
RobotController	2	пройдены	формирование результирующего управляющего воздействия, поведение при нулевой команде

Как следует из таблицы, общий объём модульных испытаний составляет 51 тест, объединённый в 9 тестовых наборах. По результатам выполнения все тесты успешно завершены. Это позволяет сделать вывод о корректности реализации выделенных программных компонентов в пределах предусмотренных тестовых сценариев и о согласованности текущей версии программного кода с принятыми условиями проверки.

Успешное прохождение тестов для компонентов RobotEKF, Odometry, PID, FFModel, IMU, Motor, Encoder, Robot и RobotController свидетельствует о корректности основных вычислительных алгоритмов, процедур обработки измерений, формирования состояния платформы и верхнеуровневого вычисления управляющего воздействия. Существенно, что тестовый набор охватывает не только штатные режимы работы, но и граничные случаи, в том числе нулевой шаг дискретизации, насыщение управляющего воздействия, изменение заданного значения, нештатные состояния аппаратного интерфейса и восстановление после отказа. Тем самым результаты модульных испытаний показывают работоспособность отдельных компонентов не только в нормальных условиях функционирования, но и в ситуациях, критичных с точки зрения надёжности системы управления.

С инженерной точки зрения модульное тестирование в данной работе выполняет не только функцию подтверждения корректности уже реализованных компонентов, но и функцию сопровождения разработки. Наличие формализованного набора тестов позволяет выполнять регрессионную проверку после внесения изменений в код, контролировать согласованность поведения отдельных подсистем и своевременно выявлять возможные отклонения в логике работы компонентов. Это особенно важно для системы управления, в которой вычислительные модули, обработка датчиков и исполнительная логика тесно связаны между собой, но при этом должны сохранять независимость на уровне архитектуры [10, 12, 19].

Таким образом, проведённые модульные испытания свидетельствуют о работоспособности и корректности реализации отдельных компонентов системы управления в пределах реализованного набора тестовых сценариев и обосновывают переход к следующему этапу проверки — испытаниям полной системы в среде моделирования, где уже оценивается совместная работа реализованных модулей в составе единого контура управления.

3.5.2. Тестирование в среде моделирования

Тестирование программного обеспечения системы управления ходовой частью мобильного робота выполнено в среде моделирования по схеме программного моделирования в контуре. В рамках данного подхода вычислительная часть системы управления исполняется в вычислительной среде общего назначения, тогда как объект управления представлен виртуальной моделью мобильной платформы в среде Webots. Такой способ испытаний позволяет исследовать поведение замкнутого контура управления без использования целевой аппаратной платформы, но при сохранении взаимодействия с датчиками и исполнительными механизмами через программные адаптеры среды моделирования [13, 18].

В отличие от модульного тестирования, на данном этапе рассматривается система в целом, включая обновление данных датчиков, формирование текущего

состояния платформы, вычисление управляющего воздействия и передачу управляющего сигнала на исполнительные механизмы. Тем самым испытания в среде моделирования позволяют оценить корректность работы полного дискретного замкнутого контура управления и получить количественные показатели качества функционирования системы. Для этого в программе реализовано ведение журнала телеметрии, содержащего время, заданные и фактические линейную и угловую скорости, ошибки регулирования, управляющие напряжения левого и правого моторов, а также координаты положения робота [1, 7, 13, 18].

3.5.2.1. Интеграция системы управления со средой моделирования

Интеграция программной системы со средой моделирования реализована на уровне аппаратной абстракции. В рамках данного подхода аппаратно-зависимые компоненты заменяются реализациями, взаимодействующими с программным интерфейсом среды Webots, при сохранении неизменной вычислительной логики системы управления. В контроллере моделирования создаются реализации интерфейсов аппаратной абстракции для инерциального измерительного модуля, энкодеров левого и правого колёс, а также исполнительных механизмов приводов. Затем на их основе конструируются объекты IMU, Encoder, Motor, далее компонент Robot, модельный модуль FFMModel, два ПИД-регулятора и компонент RobotController. Такое построение означает, что в среде моделирования используется тот же вычислительный контур управления, что и в основной программной системе, а замене подвергается только нижний уровень взаимодействия с внешней средой [10, 12, 13, 18].

Значения датчиков формируются на основе состояния виртуальной модели и передаются в систему управления через интерфейсы аппаратного уровня. В свою очередь, вычисленные системой управления воздействия преобразуются в сигналы управления виртуальными приводами. Дополнительно используется модуль GPS, включаемый как источник опорных данных о

линейной скорости и координатах робота. Именно по этим данным в журнал телеметрии записываются фактическая линейная скорость и положение платформы. Фактическая угловая скорость при этом фиксируется по внутреннему состоянию робота, формируемому программной системой управления [18].

Таким образом, среда моделирования в рассматриваемой работе используется не как отдельная упрощённая модель, а как полноценная платформа исполнения основного контура управления. Это позволяет оценивать поведение именно разработанного программного обеспечения, а не специально созданной экспериментальной схемы [13, 18].

Схема интеграции системы управления со средой моделирования представлена на рисунке 10.

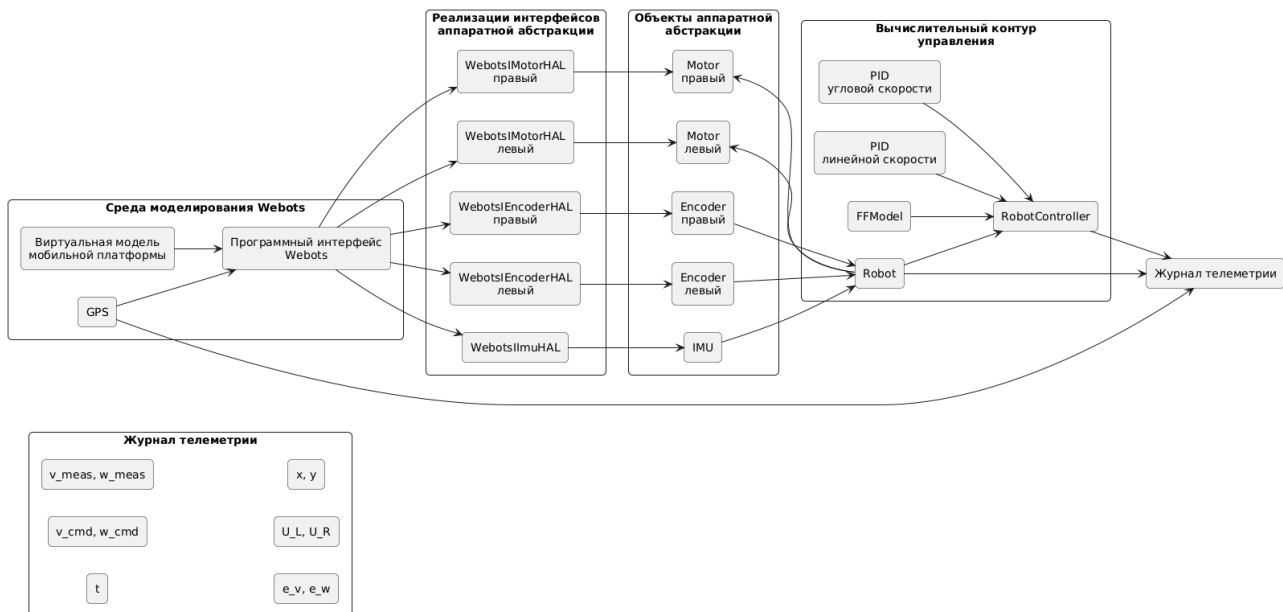


Рисунок 10 — Схема интеграции системы управления со средой моделирования

3.5.2.2. Методика экспериментальной оценки

Оценка качества управления выполняется на основе сравнения заданных и фактических параметров движения мобильной платформы. В качестве основных анализируемых величин используются линейная и угловая скорости робота. Для каждого шага моделирования формируется запись телеметрии, содержащая заданные и фактические значения этих скоростей, ошибки

регулирования, управляющие напряжения на приводах, а также координаты робота. На основе накопленной телеметрии затем вычисляются сводные показатели качества управления [1, 7].

Ошибка по линейной скорости определяется выражением

$$e_v(t) = v_{\text{зад}}(t) - v(t)$$

ошибка по угловой скорости — выражением

$$e_\omega(t) = \omega_{\text{зад}}(t) - \omega(t)$$

В дискретной форме для оценки качества управления используются следующие показатели.

Средняя абсолютная ошибка:

$$\bar{e}_v = \frac{1}{N} \sum_{k=1}^N |e_{v,k}|$$

$$\bar{e}_\omega = \frac{1}{N} \sum_{k=1}^N |e_{\omega,k}|$$

Среднеквадратическая ошибка:

$$e_{v,RMS} = \sqrt{\frac{1}{N} \sum_{k=1}^N e_{v,k}^2}$$

$$e_{\omega,RMS} = \sqrt{\frac{1}{N} \sum_{k=1}^N e_{\omega,k}^2}$$

Максимальная абсолютная ошибка:

$$e_{v,max} = \max(|e_{v,k}|)$$

$$e_{\omega,max} = \max(|e_{\omega,k}|)$$

Для составных и замкнутых траекторий дополнительно используется ошибка конечного положения:

$$e_p = \sqrt{(x_T - x_0)^2 + (y_T - y_0)^2}$$

В приведённых выражениях N обозначает число отсчётов телеметрии на интервале наблюдения, k — номер отсчёта, x_0, y_0 — начальные координаты робота, а x_T, y_T — координаты в конце рассматриваемого движения.

Выбранный набор показателей позволяет охарактеризовать систему как в переходных процессах, так и в установившемся режиме. Средняя абсолютная ошибка отражает типичный уровень отклонений, среднеквадратическая ошибка характеризует интегральное влияние отклонений на всём интервале наблюдения, максимальная ошибка показывает наиболее неблагоприятный момент поведения системы, а ошибка конечного положения — накопленный геометрический результат движения на длительной или составной траектории [2].

При интерпретации результатов необходимо учитывать особенность используемой телеметрии. Линейная скорость оценивается по внешнему опорному источнику, а именно по данным GPS среды моделирования. Угловая скорость при этом фиксируется по внутреннему состоянию робота, формируемому программной системой управления, а не по независимому внешнему измерителю. Следовательно, результаты по линейной скорости опираются на внешний источник данных, тогда как результаты по угловой скорости в большей степени характеризуют согласованность и устойчивость внутреннего оценочно-управляющего контура. Это обстоятельство не снижает значимости проведённых испытаний, однако должно учитываться при интерпретации показателей углового канала [18].

В рамках данной работы результат испытания считается удовлетворительным, если система сохраняет устойчивость, не демонстрирует нарастающих колебаний, обеспечивает малые ошибки в установившемся режиме и воспроизводит заданный характер движения без разрушения требуемой структуры траектории. Для составных траекторий дополнительным критерием является ограниченность накопленной ошибки конечного положения [1, 2].

Структура формирования телеметрии и расчёта показателей качества приведена на рисунке 11.

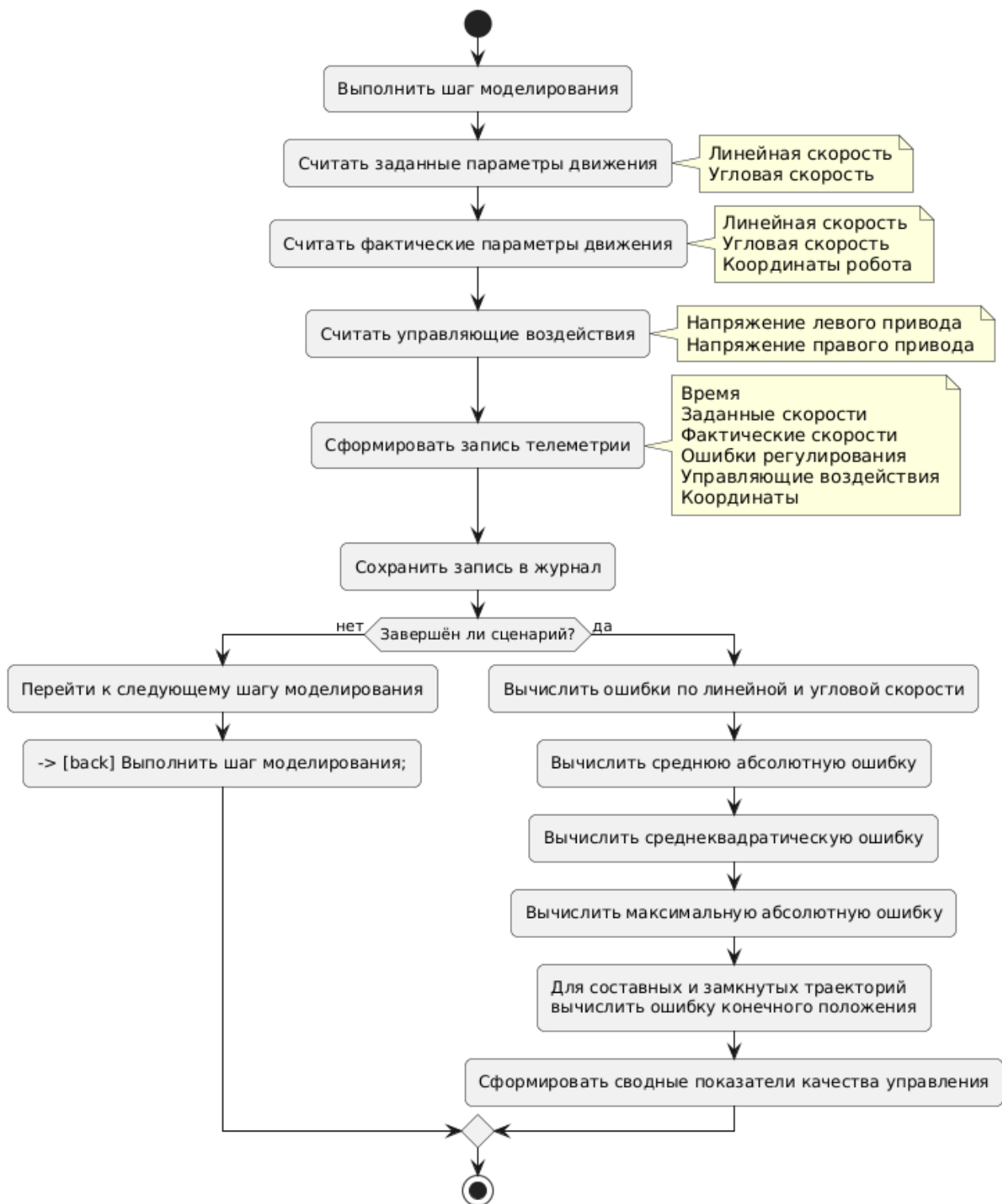


Рисунок 11 — Структура формирования телеметрии и расчёта показателей качества

3.5.2.3. Результаты тестирования

В рамках экспериментальной проверки в среде моделирования рассматриваются четыре характерных сценария: прямолинейное движение,

поворот на месте, движение по окружности и движение по квадратной траектории. Такой набор испытаний позволяет последовательно оценить работу системы в простых установившихся режимах, в режиме чистого вращения, при одновременном поддержании линейной и угловой скорости, а также в условиях многократной смены режима движения [5, 8, 11].

Первые два сценария предназначены для отдельной проверки каналов линейного и углового управления. Сценарий движения по окружности позволяет оценить согласованную работу обоих каналов в длительном установившемся режиме. Сценарий движения по квадратной траектории используется для анализа поведения системы в условиях повторяющихся переходных процессов и накопления ошибки на составной траектории [1, 5, 7, 11].

3.5.2.3.1. Прямолинейное движение

В данном сценарии задаётся постоянная линейная скорость при отсутствии вращения:

$$v_{зад} = 0,2 \text{ м/с}$$

$$\omega_{зад} = 0 \text{ рад/с}$$

Данный сценарий используется для изолированной оценки канала управления линейной скоростью. Помимо точности поддержания поступательного движения он позволяет оценить отсутствие заметного паразитного вращения платформы, поскольку в идеальном случае при прямолинейном движении угловая скорость должна оставаться близкой к нулю [5, 11].

На рисунке 12 представлена зависимость заданной и фактической линейной скорости от времени.

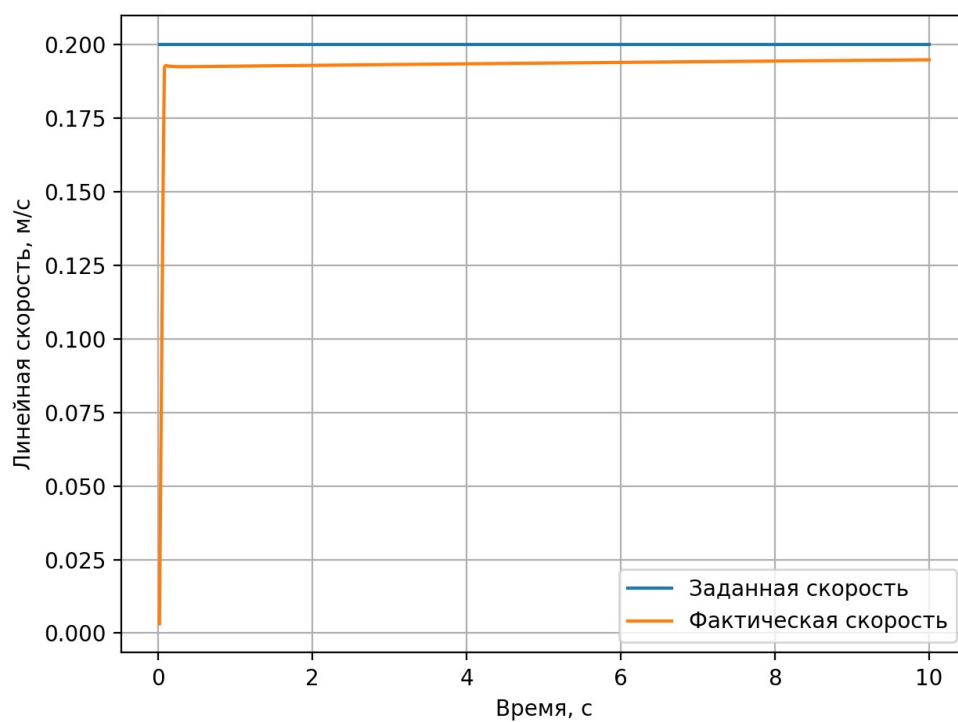


Рисунок 12 — Зависимость заданной и фактической линейной скорости от времени

На рисунке 13 представлена ошибка по линейной скорости.

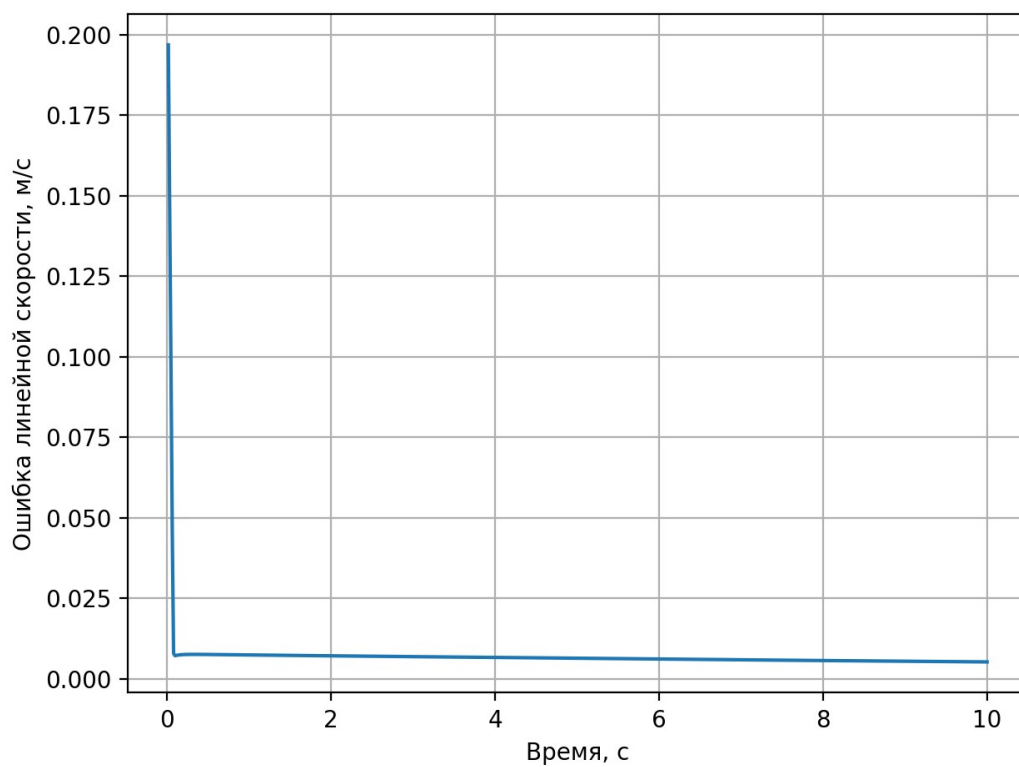


Рисунок 13 — Ошибка по линейной скорости при прямолинейном движении

Результаты количественной оценки имеют вид:

$$\bar{e}_v = 0,007091 \text{ м/с}$$

$$e_{v,max} = 0,196803 \text{ м/с}$$

$$e_{v,RMS} = 0,012497 \text{ м/с}$$

В начальный момент времени наблюдается переходный процесс, обусловленный разгоном системы из состояния покоя. Именно этим объясняется максимальное значение ошибки в начале эксперимента. После завершения переходного процесса фактическая линейная скорость стабилизируется вблизи заданного значения, а ошибка уменьшается до малых величин. Следовательно, канал управления поступательным движением обеспечивает корректное поддержание требуемой скорости в установившемся режиме.

Дополнительно данный сценарий показывает, что в процессе прямолинейного движения не выявляется заметного паразитного вращения, способного привести к отклонению траектории от прямой линии. Это означает, что система в целом сохраняет согласованность работы левого и правого приводов при реализации симметричного режима движения.

3.5.2.3.2. Поворот на месте

В данном сценарии исследуется режим чистого вращательного движения платформы при отсутствии поступательного перемещения:

$$v_{зад} = 0, \quad \omega_{зад} = 1,0 \text{ рад/с}$$

Смысл данного теста состоит в изолированной проверке канала управления угловой скоростью. Одновременно он позволяет оценить отсутствие заметного паразитного поступательного перемещения центра платформы, поскольку в идеальном случае робот должен вращаться на месте без существенного линейного смещения.

На рисунке 14 представлена зависимость заданной и фактической угловой скорости от времени.

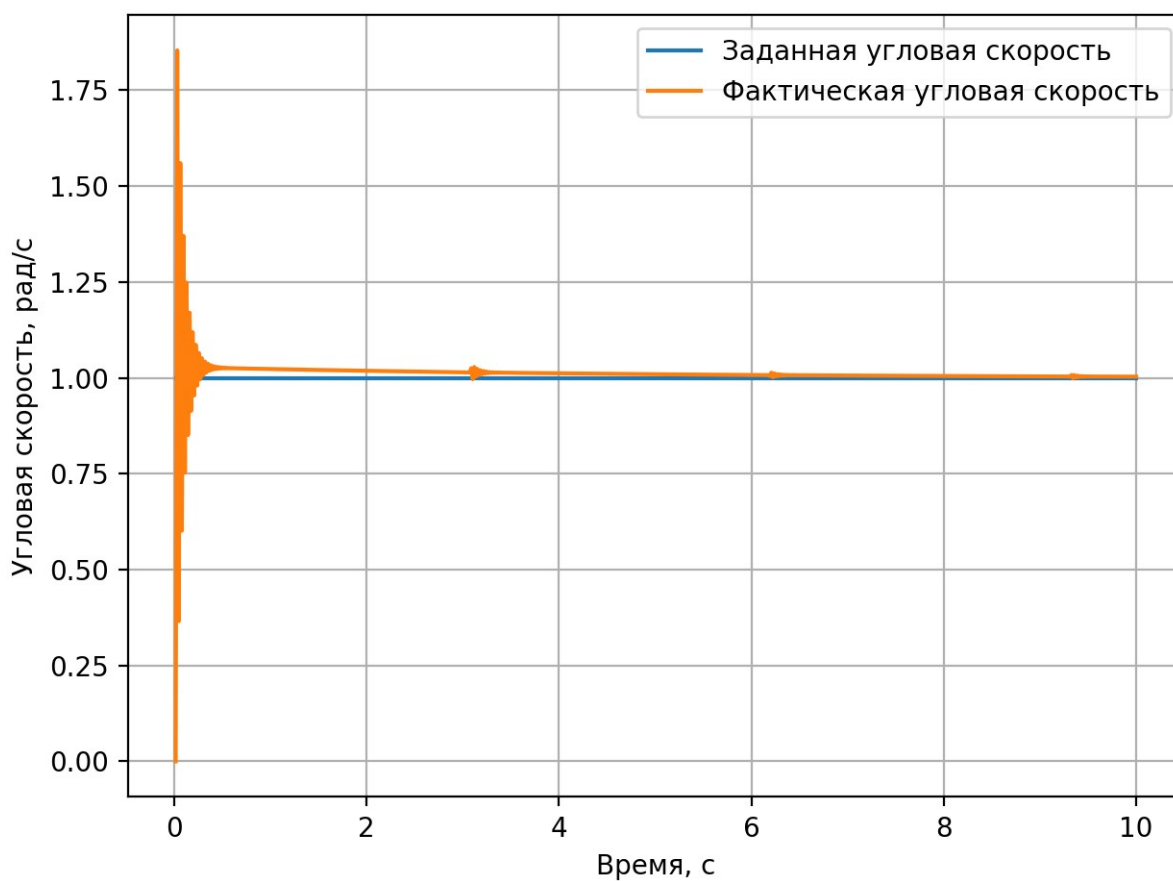


Рисунок 14 — Зависимость заданной и фактической угловой скорости при повороте на месте

На рисунке 15 представлена ошибка по угловой скорости.

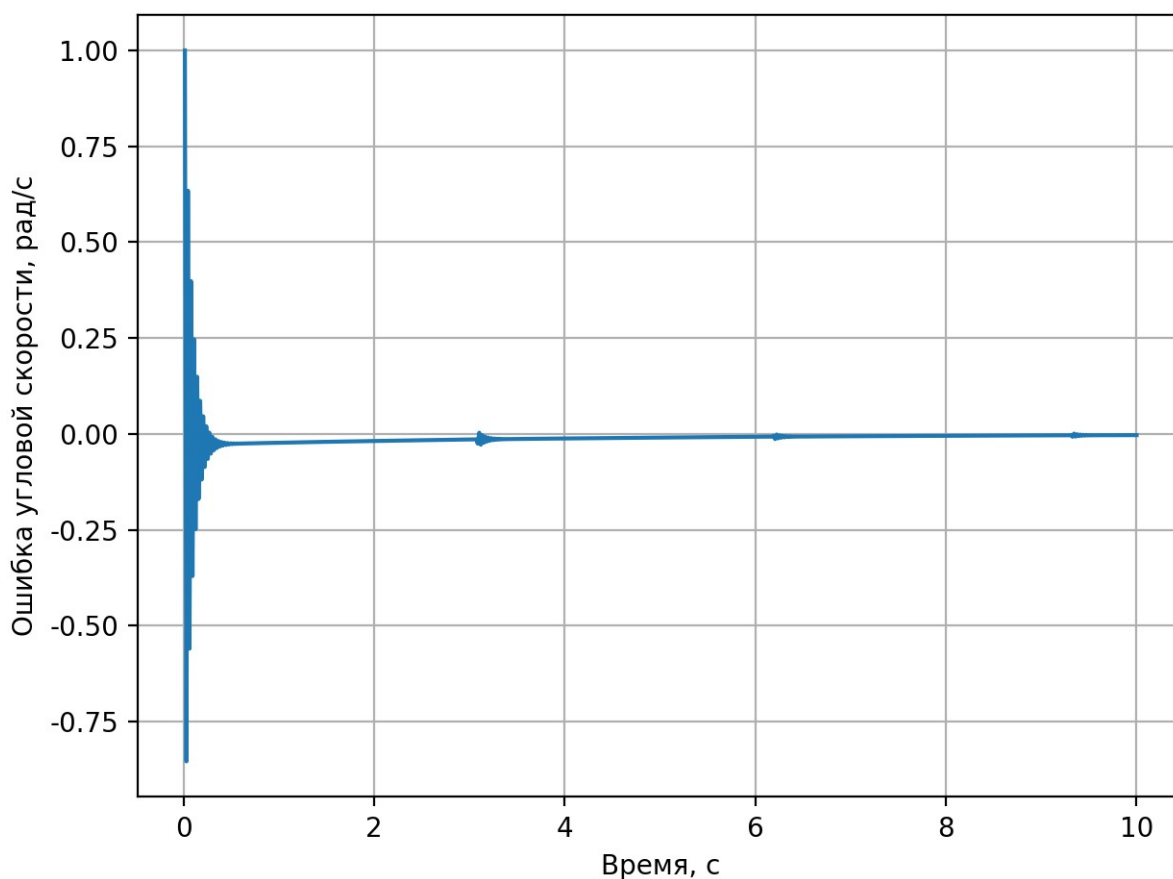


Рисунок 15 — Ошибка по угловой скорости при повороте на месте

Результаты количественной оценки имеют вид:

$$\bar{e}_{\omega}=0,018865 \text{ рад/с}, e_{\omega,max}=1,0 \text{ рад/с}, e_{\omega,RMS}=0,069911 \text{ рад/с}$$

Полученные результаты показывают, что в начальный момент времени система проходит переходный процесс, обусловленный разгоном из состояния покоя. После его завершения фактическая угловая скорость стабилизируется вблизи заданного значения. Максимальная ошибка достигается в начальный момент, когда задание скачкообразно становится ненулевым, а фактическая скорость ещё равна нулю. В установившемся режиме отклонения остаются малыми, что свидетельствует о корректной работе канала управления угловой скоростью.

Дополнительно данный сценарий показывает, что при вращении не выявляется существенного паразитного поступательного движения. Следовательно, система обеспечивает согласованную работу приводов в режиме

встречного вращения колёс и корректно реализует поворот платформы вокруг собственной вертикальной оси.

3.5.2.3.3. Движение по окружности

В данном сценарии исследуется движение по окружности фиксированного радиуса при постоянных линейной и угловой скоростях. Формально такой режим соответствует одновременной работе каналов управления линейным и угловым движением в установившемся режиме:

$$v_{\text{зад}} = 0,2 \text{ м/с}, \quad \omega_{\text{зад}} = 0,04 \text{ рад/с}, \quad R = \frac{v}{\omega} = 5 \text{ м}$$

Этот сценарий позволяет оценить не только точность поддержания каждой из скоростей по отдельности, но и их взаимную согласованность, поскольку именно отношение линейной и угловой скорости определяет радиус движения.

На рисунке 16 представлена траектория движения робота.

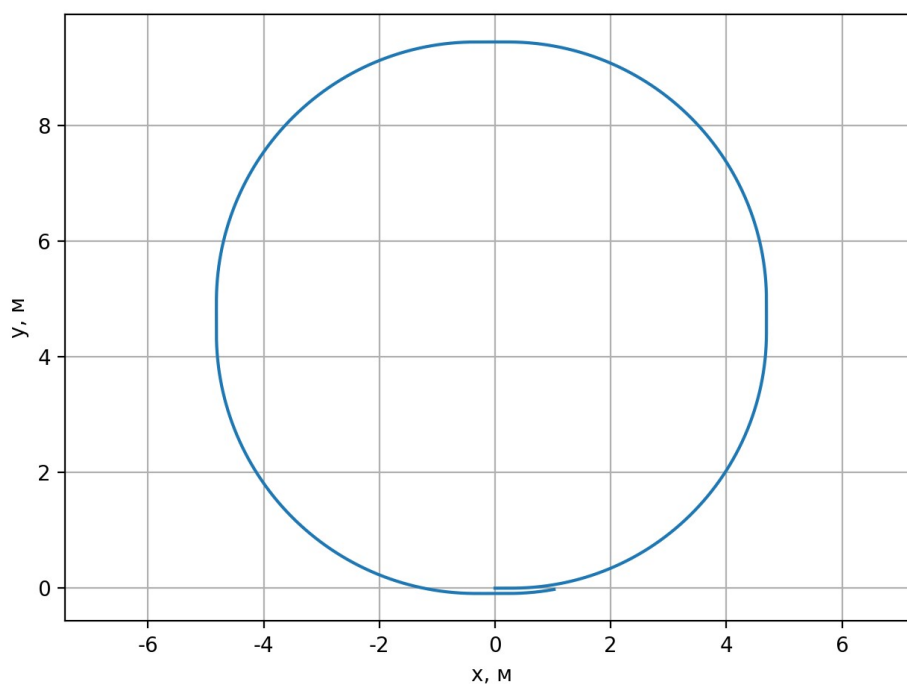


Рисунок 16 — Траектория движения робота при движении по окружности

На рисунках 17 и 18 представлены зависимости линейной и угловой скорости от времени.

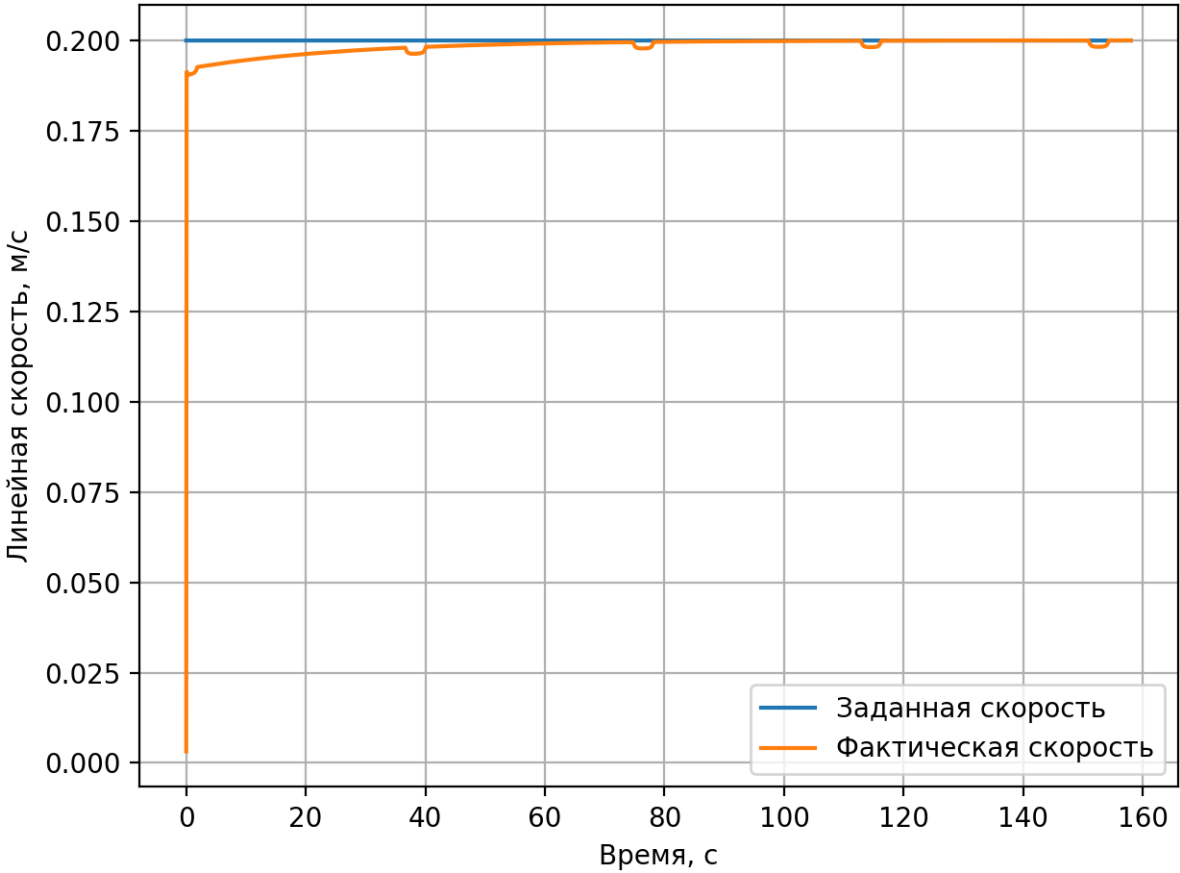


Рисунок 17 — Зависимости линейной при движении по окружности

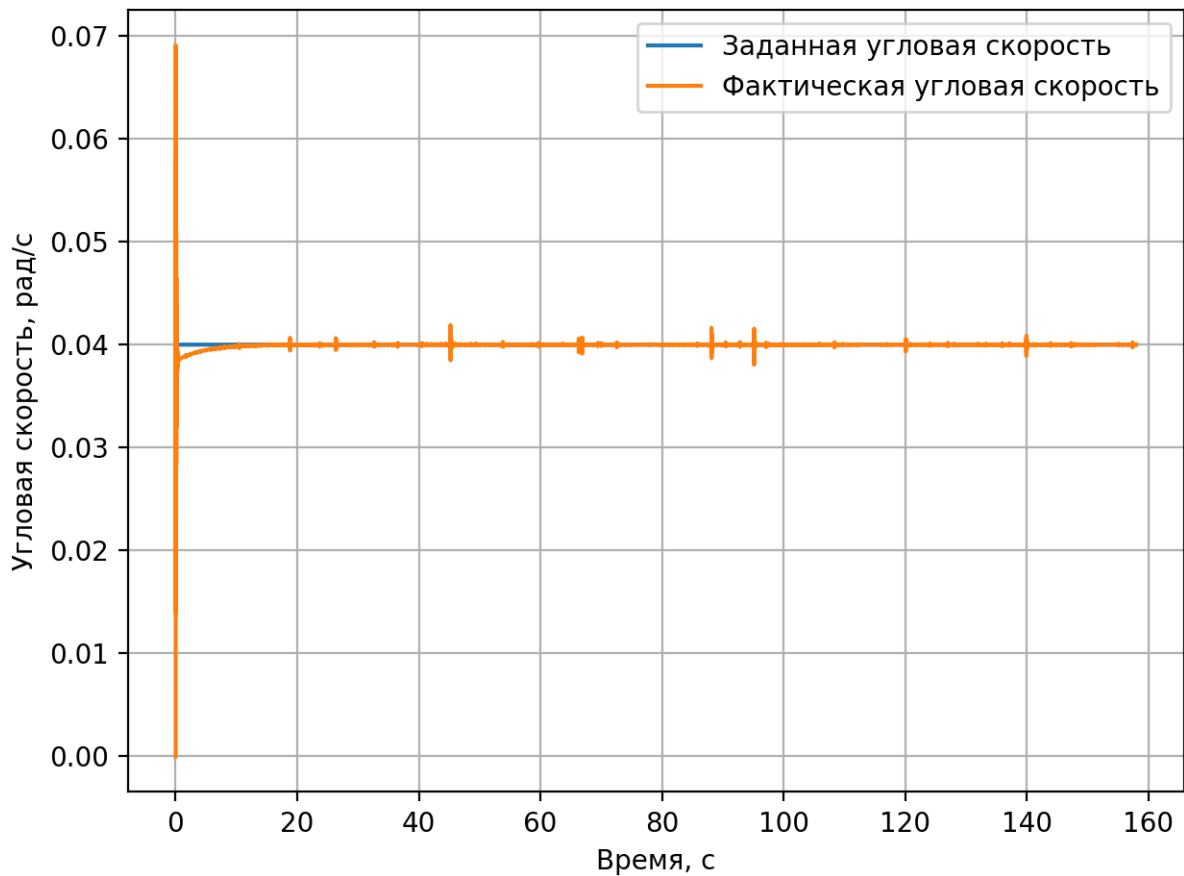


Рисунок 18 — Зависимости угловой скорости при движении по окружности

Результаты количественной оценки имеют вид:

$$\bar{e}_v = 0,006049 \text{ м/с}, \quad \bar{e}_\omega = 0,000491 \text{ рад/с}, \quad e_p = 3,76 \text{ м}$$

Полученные значения средней ошибки по линейной и угловой скорости указывают на высокую точность поддержания заданных скоростей в процессе движения. Это означает, что в локальном смысле каналы управления работают корректно и устойчиво. Однако при длительном движении по замкнутой траектории возникает заметная ошибка конечного положения. Следовательно, локально малые ошибки по скоростям не гарантируют точного замыкания траектории при длительном движении.

Данный результат объясняется тем, что система управления в рассматриваемой постановке не содержит обратной связи по глобальному

положению робота и не выполняет коррекцию траектории в мировых координатах. Поэтому даже небольшие отклонения мгновенных скоростей, интегрируясь во времени, приводят к накоплению заметной ошибки положения. Для скоростного контура управления такой характер ошибки является ожидаемым по своей природе, однако полученная величина накопленного отклонения показывает, что на длительных замкнутых траекториях без позиционной коррекции геометрическая точность движения остаётся ограниченной.

3.5.2.3.4 Движение по квадратной траектории

Сценарий движения по квадратной траектории используется для проверки работы системы управления в условиях многократной смены режима движения. В рамках данного эксперимента прямолинейные участки чередуются с поворотами на угол, близкий к $\pi/2$, что создаёт последовательность повторяющихся переходных процессов.

Данный сценарий имеет особое значение, поскольку позволяет оценить не только поддержание линейной и угловой скорости в отдельных режимах, но и способность системы сохранять устойчивость, согласованность и ограниченность накопленной ошибки в условиях составного манёвра. Именно поэтому тест на квадратную траекторию является наиболее показательным для практической оценки системы управления.

На рисунке 19 представлена траектория движения робота.

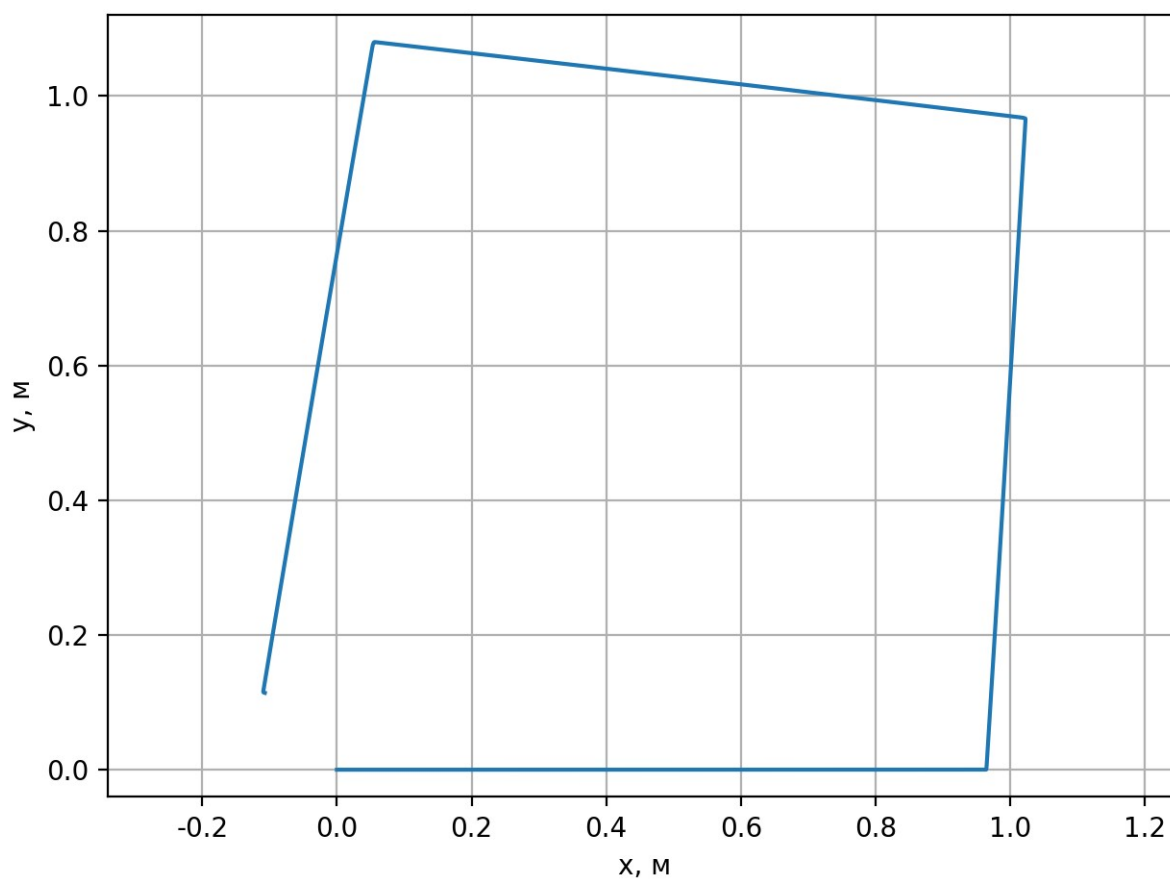


Рисунок 19 — Траектория движения робота при движении по квадратной траектории

На рисунках 20 и 21 показаны ошибки линейной и угловой скорости.

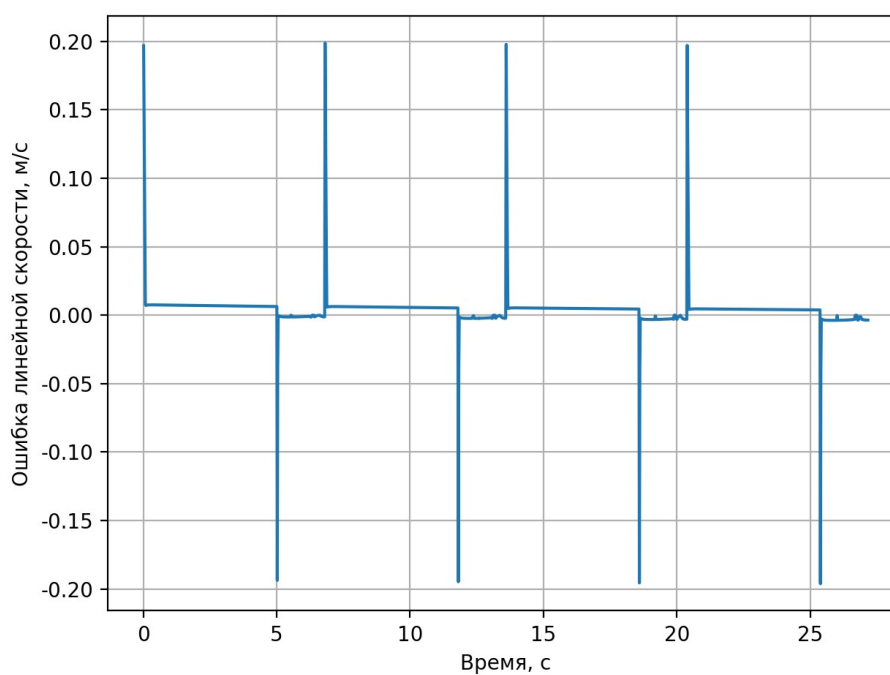


Рисунок 20 — Ошибка линейной при движении по квадратной траектории

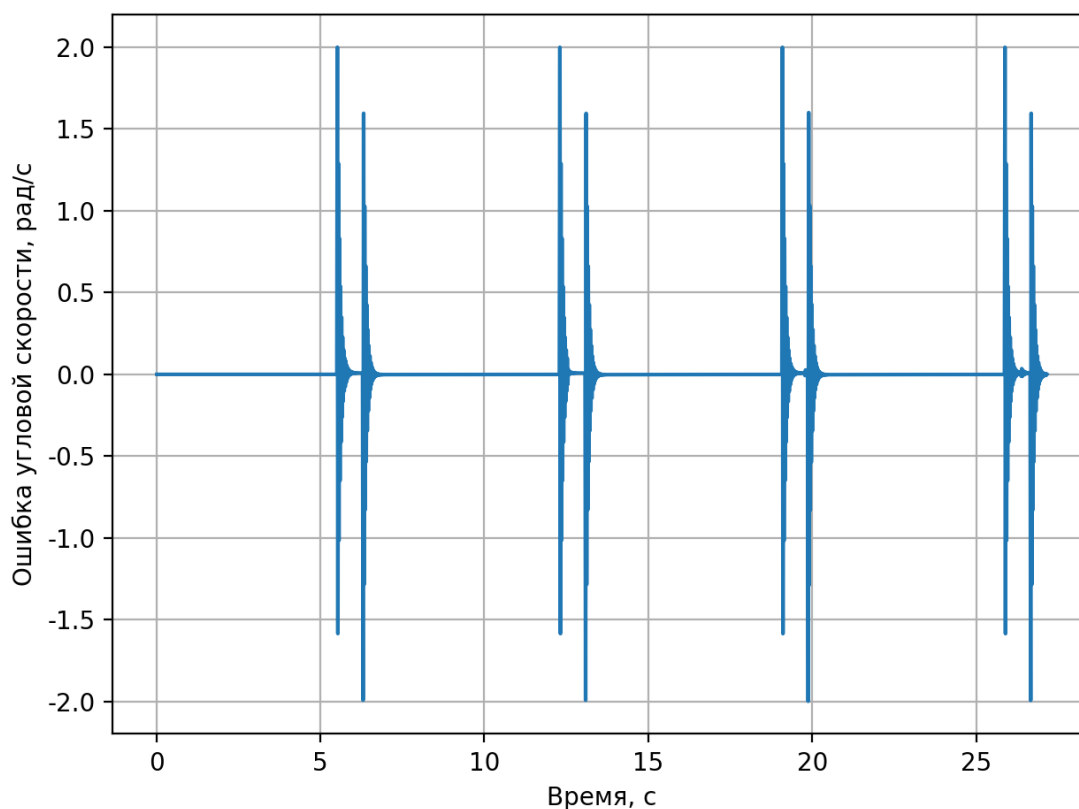


Рисунок 21 — Ошибка угловой скорости при движении по квадратной траектории

Результаты количественной оценки имеют вид:

$$\bar{e}_v = 0,006227 \text{ м/с}, \quad \bar{e}_\omega = 0,048406 \text{ рад/с}, \quad e_p = 0,156 \text{ м}$$

По сравнению с более простыми сценариями данный режим является существенно более чувствительным к ошибкам управления, поскольку локальные отклонения, возникающие в моменты переключения между прямолинейным движением и поворотом, накапливаются и влияют на последующие участки траектории. При этом фактическая траектория в целом сохраняет форму заданного маршрута, а ошибка конечного положения остаётся сравнительно малой.

Это позволяет сделать вывод о корректной работе замкнутого контура управления в условиях составного манёвра. Несмотря на наличие переходных процессов и локального увеличения ошибок в моменты смены режима, система не теряет устойчивости, не демонстрирует нарастающих колебаний и

обеспечивает воспроизведение общей геометрии траектории при ограниченном накоплении ошибки.

Для сопоставления результатов всех проведённых экспериментов сводные показатели качества управления приведены в таблице 2.

Таблица 2 — Сводные результаты испытаний в среде моделирования

Сценарий	$\bar{e}_v$, м/с	$\bar{e}_\omega$, рад/с	$e_{v,max}$, м/с	$e_{\omega,max}$, рад/с	$e_{v,RMS}$, м/с	$e_{\omega,RMS}$, рад/с	e_p , м
Прямолинейное движение	0,007091	—	0,196803	—	0,012497	—	—
Поворот на месте	—	0,018865	—	1,0	—	0,069911	—
Движение по окружности	0,006049	0,000491	—	—	—	—	3,76
Движение по квадратной траектории	0,006227	0,048406	—	—	—	—	0,156

Сводные результаты испытаний показывают, что разработанная система управления обеспечивает малые ошибки по линейной и угловой скорости в установившихся режимах движения. Наибольшие отклонения наблюдаются в переходных процессах, что соответствует характеру дискретной системы управления скоростями. Для длительных и составных траекторий проявляется накопление ошибки конечного положения, обусловленное отсутствием обратной связи по глобальным координатам робота. При этом во всех рассмотренных сценариях система сохраняет устойчивость и воспроизводит требуемый режим движения без нарастающих колебаний.

3.5.2.4 Выводы по испытаниям в среде моделирования

Проведённые испытания по схеме программного моделирования в контуре показывают, что разработанная система управления корректно функционирует в составе полного замкнутого контура и обеспечивает устойчивое воспроизведение заданных режимов движения в среде моделирования. Простые сценарии прямолинейного движения и поворота на месте свидетельствуют о

корректной работе каналов управления линейной и угловой скоростью, а также об отсутствии существенных паразитных движений в ортогональном канале. Более сложные сценарии движения по окружности и квадратной траектории показывают, что система сохраняет работоспособность и в условиях длительного движения и многократной смены режима.

Количественные показатели качества показывают, что основная доля ошибок приходится на переходные процессы, возникающие в моменты начала движения и переключения между различными режимами. В установившемся режиме отклонения по линейной и угловой скорости остаются малыми. При этом на составных и длительных траекториях наблюдается накопление ошибки положения, что является естественным следствием выбранной структуры управления, ориентированной на регулирование скоростей, а не на коррекцию глобальных координат робота.

Таким образом, тестирование в среде моделирования свидетельствует о работоспособности разработанного программного обеспечения в составе полного контура программного моделирования и показывает, что система управления обеспечивает приемлемое качество движения мобильной платформы в рамках поставленной задачи.

3.6. Выводы

В ходе тестирования разработанной программы выполнена двухуровневая проверка её работоспособности. На первом уровне проведено модульное тестирование, охватывающее основные вычислительные, измерительные и управляющие компоненты системы. Набор модульных испытаний включает 9 тестовых групп и 51 отдельное испытание, по результатам выполнения которых подтверждена корректность реализации ключевых программных модулей и их устойчивость в пределах предусмотренных тестовых сценариев.

На втором уровне выполнены испытания в среде моделирования по схеме программного моделирования в контуре, позволяющие проверить работу

полного замкнутого контура управления. В рамках экспериментальной проверки рассмотрены четыре характерных сценария: прямолинейное движение, поворот на месте, движение по окружности и движение по квадратной траектории. Полученные результаты показывают, что система управления обеспечивает корректное воспроизведение заданных режимов движения, сохраняет устойчивость и не демонстрирует нарастающих колебаний в рассмотренных режимах.

Проведённые испытания показали, что наибольшие отклонения возникают в переходных процессах, тогда как в установившихся режимах ошибки линейной и угловой скорости остаются малыми. Для длительных и составных траекторий выявляется накопление ошибки конечного положения, что обусловлено отсутствием обратной связи по глобальным координатам робота и соответствует принятой структуре скоростного управления. При этом в целом система сохраняет согласованность работы каналов управления и приемлемое качество движения мобильной платформы.

Таким образом, результаты тестирования подтверждают работоспособность разработанного программного обеспечения, корректность реализации его основных компонентов и пригодность выбранной структуры управления к практическому использованию в составе системы управления ходовой частью мобильного робота. Полученные результаты также создают основу для последующего этапа работы, связанного с переносом программы на целевую платформу STM32.

4. Особенности реализации на платформе STM32

4.1 Общие положения

Разработанная программа управления ходовой частью четырёхколёсного робота изначально проектировалась с учётом её последующего применения на встраиваемой микроконтроллерной платформе семейства STM32. Такая направленность отражена как в архитектуре программного обеспечения, так и в организации проекта в целом. Помимо подсистем моделирования, тестирования и основных компонентов системы управления, в составе проекта предусмотрены средства сборки целевой прошивки для ARM-платформы, что позволяет рассматривать разработанную систему не только как модельный программный прототип, но и как основу для последующего развёртывания на микроконтроллере [15, 17, 21].

При этом в рамках настоящей работы основная экспериментальная проверка корректности алгоритмов управления выполняется в программной среде по схеме программного моделирования замкнутого контура. Такой подход позволяет исследовать свойства разработанной системы, проверить корректность вычислительных процедур и оценить качество регулирования в контролируемых условиях без необходимости завершения полной аппаратной интеграции [13]. По этой причине в данной главе рассматриваются не результаты натурных испытаний на физическом образце робота, а особенности переноса и адаптации разработанной программы к целевой платформе STM32.

С инженерной точки зрения перенос программы на микроконтроллерную платформу основывается на принципе разделения платформенно-независимых и платформенно-зависимых компонентов системы. Подсистема управляющей логики, вычислительная подсистема управления и подсистема аппаратной абстракции формируют основу программной структуры, сохраняемую при переходе между различными средами исполнения. Изменения касаются прежде всего платформенно-зависимой подсистемы, реализующей взаимодействие с

аппаратными ресурсами конкретной микроконтроллерной платформы [10, 12, 22].

Такое построение программы имеет принципиальное значение, поскольку позволяет сохранить результаты, полученные на этапах проектирования, отладки и тестирования вычислительной части системы, при переходе к встраиваемой реализации. Иначе говоря, адаптация к STM32 в данной работе не рассматривается как повторная разработка системы управления, а понимается как реализация платформенно-зависимой подсистемы, обеспечивающей выполнение уже разработанного алгоритма в условиях микроконтроллерной среды [9, 10, 12, 22].

Таким образом, задача адаптации программы к платформе STM32 сводится к реализации взаимодействия с периферийными устройствами микроконтроллера, организации детерминированного дискретного цикла управления и включению вычислительных модулей в состав прошивки целевого устройства [9, 15, 20, 21, 22]. Именно эти вопросы составляют содержание настоящей главы. На рисунке 22 представлена общая схема адаптации программы управления к платформе STM32.

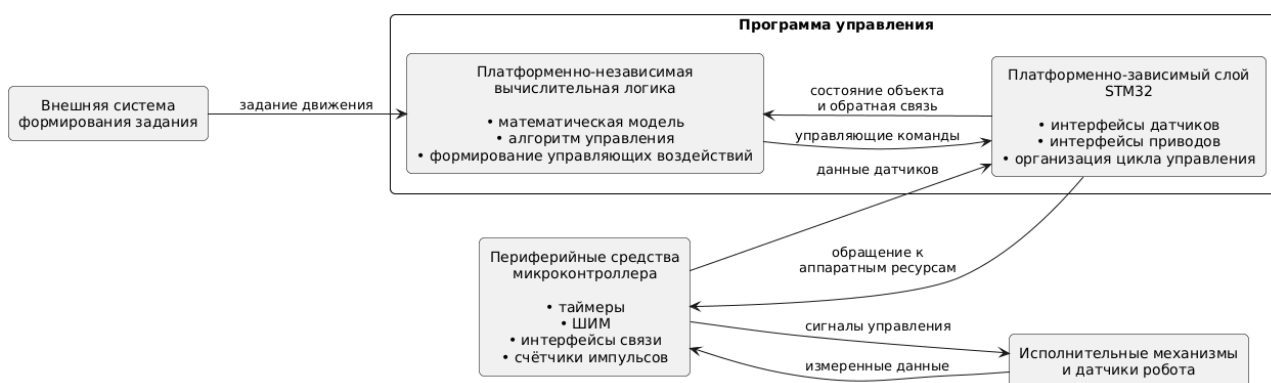


Рисунок 22 — Общая схема адаптации программы управления к платформе STM32

4.2 Признаки ориентации проекта на платформу STM32

Ориентация разработанного проекта на платформу STM32 подтверждается его структурой, составом программных модулей и организацией

системы сборки. В корневой структуре проекта выделены подсистемы `robot`, `tests`, `firmware` и `simulation`. Такое построение показывает, что целевая прошивка рассматривается как органическая часть общего программного комплекса, а не как внешнее дополнение к средству моделирования или тестирования. Иначе говоря, проект изначально формировался с учётом существования как моделируемой, так и встраиваемой реализации.

Существенным признаком такой ориентации является наличие отдельного раздела `firmware`, в рамках которого формируется исполняемый файл `firmware.elf`. При его построении используются вычислительные библиотеки `robot` и `robotmath`, а также платформенно-зависимый модуль `stm32_platform`. После этапа компоновки дополнительно создаются файлы `firmware.bin` и `firmware.hex`, что соответствует типовой практике подготовки прошивки для микроконтроллерных устройств. Тем самым проект поддерживает не только сборку библиотек и промежуточных модулей, но и формирование итогового загрузочного образа [9, 15, 17].

Дополнительным подтверждением направленности проекта на STM32 является наличие специализированных конфигураций сборки. В системе сборки предусмотрена отдельная конфигурация `arm`, использующая файл настройки инструментария кросс-компиляции, а также конфигурация `armware`, предназначенная для формирования исполняемого образа прошивки под целевую архитектуру ARM. Наличие таких конфигураций свидетельствует о том, что поддержка микроконтроллерной платформы заложена в проект на уровне его базовой инфраструктуры сборки, а не вводится как вспомогательное или временное решение [17].

Отдельного внимания заслуживает присутствие файла компоновки `STM32F411CEUx.ld`, в котором задаются параметры распределения кода и данных в памяти микроконтроллера. Использование специализированного файла компоновки означает, что проект адаптирован к конкретной целевой аппаратной платформе не только на уровне компиляции исходных текстов, но и на уровне

формирования конечного исполняемого образа с учётом особенностей адресного пространства устройства [15, 20, 21].

Таким образом, проект уже содержит совокупность реальных признаков ориентации на платформу STM32: выделенный раздел прошивки, платформенно-зависимый модуль, конфигурации кросс-компиляции и специализированный файл компоновки. Вместе с тем необходимо различать готовность программного проекта к сборке под целевую архитектуру и завершённость его экспериментальной проверки на физической аппаратной платформе. В рамках настоящей работы первая из этих задач решена на уровне структуры проекта и средств сборки, тогда как вторая требует отдельного этапа аппаратной интеграции и натурной отладки.

Таблица 3 — Признаки ориентации проекта на платформу STM32

Элемент проекта	Назначение
firmware	формирование целевой прошивки
robot, robotmath	вычислительные библиотеки, подключаемые к прошивке
stm32_platform	модуль платформенно-зависимой подсистемы для микроконтроллера
arm	конфигурация сборки под ARM
armware	конфигурация сборки исполняемого образа прошивки
STM32F411CEUx.ld	задание параметров компоновки под целевую платформу

Следовательно, дальнейшее изложение целесообразно сосредоточить не на обосновании самой возможности сборки проекта под платформу STM32, а на рассмотрении тех программных решений, которые обеспечивают выполнение алгоритма управления в условиях микроконтроллерной среды.

4.3 Требования к целевой платформе

Микроконтроллерная платформа семейства STM32 относится к классу встраиваемых вычислительных систем, для которых характерны ограниченность

вычислительных ресурсов, конечный объём оперативной и постоянной памяти, а также повышенные требования к предсказуемости выполнения программного кода. В связи с этим реализация алгоритмов управления на такой платформе должна учитывать не только корректность вычислений, но и возможность их устойчивого выполнения в пределах заданного периода дискретизации [9, 15, 21].

В отличие от среды выполнения на персональном компьютере, где вычислительные ресурсы в рамках рассматриваемой задачи практически не накладывают существенных ограничений на структуру программы, в микроконтроллерной системе определяющим становится требование детерминированного выполнения всех этапов цикла управления. Это означает, что чтение данных с датчиков, вычисление текущего состояния, формирование управляющего воздействия и передача команд исполнительным механизмам должны выполняться за ограниченное и предсказуемое время [7, 9, 15].

Для рассматриваемой программы принципиальное значение имеют несколько требований. Прежде всего вычисление управляющего воздействия должно завершаться за время, меньшее периода дискретизации, поскольку нарушение этого условия приводит к срыву периодичности управления и ухудшению свойств замкнутой системы. Кроме того, доступ к датчикам и исполнительным механизмам должен быть организован с минимальными накладными расходами, так как вспомогательные операции ввода-вывода не должны занимать существенную долю времени, отведённого на выполнение управляющего цикла. Наконец, структура программы должна допускать функционирование в условиях отсутствия операционной системы общего назначения, то есть при непосредственном управлении периферийными устройствами средствами микроконтроллера [7, 9, 15, 20].

С точки зрения управления ходовой частью наибольшее значение имеют аппаратные ресурсы, обеспечивающие измерение параметров движения и формирование управляющих сигналов. К ним относятся таймеры, используемые для задания временной структуры работы программы, формирования сигналов

широотно-импульсного управления и регистрации импульсов с датчиков вращения колёс, интерфейсы обмена с инерциальными и иными датчиками, а также системные средства синхронизации, обеспечивающие периодический запуск цикла управления [20, 21, 22].

Следовательно, адаптация разработанной программы к платформе STM32 предполагает не изменение алгоритма управления как такового, а обеспечение его корректного функционирования в условиях микроконтроллерной среды за счёт реализации платформенно-зависимой подсистемы, организации периодического выполнения цикла управления и соблюдения временных ограничений, характерных для встраиваемых систем [9, 15, 20, 22].

4.4 Соответствие архитектуры программы требованиям платформы STM32

Выбранная в работе архитектура программного обеспечения обеспечивает возможность переноса разработанной системы управления на микроконтроллерную платформу STM32. Её ключевым свойством является разделение программы на подсистему управляющей логики, вычислительную подсистему управления, подсистему аппаратной абстракции и платформенно-зависимую подсистему. Благодаря такому построению верхние уровни программы не содержат прямых обращений к конкретным периферийным средствам микроконтроллера и потому сохраняют переносимость между различными средами исполнения [10, 12].

Практическое значение данного архитектурного решения состоит в том, что вычислительные библиотеки, реализующие математическую модель, обработку данных и формирование управляющих воздействий, могут быть включены в состав целевой прошивки без изменения своей внутренней структуры. Это соответствует фактической организации проекта, в которой исполняемый образ прошивки компонуется с библиотеками `robot` и `robotmath`, тогда как платформенно-зависимые функции вынесены в отдельный модуль `stm32_platform` [10, 12, 17].

Архитектурное распределение ответственности в системе может быть охарактеризовано следующим образом. Подсистема управляющей логики координирует выполнение шага управления и связывает между собой состояние робота, команду движения и вычислительные модули. Вычислительная подсистема управления реализует алгоритмы формирования управляющего воздействия и не зависит от особенностей конкретной платформы исполнения. Подсистема аппаратной абстракции задаёт единые интерфейсы взаимодействия с датчиками и исполнительными механизмами. Платформенно-зависимая подсистема реализует эти интерфейсы средствами конкретного микроконтроллера, обеспечивая связь между общей логикой программы и физическими ресурсами устройства [10, 12, 22].

Такое построение архитектуры непосредственно соответствует требованиям, предъявляемым к программному обеспечению встраиваемой системы. Во-первых, оно позволяет локализовать платформенно-зависимый код в ограниченной части проекта и тем самым упростить перенос программы на целевую аппаратную платформу. Во-вторых, оно обеспечивает повторное использование ранее разработанной и проверенной вычислительной логики. В-третьих, оно создаёт условия для поэтапной отладки системы, при которой подсистема управляющей логики и вычислительная подсистема могут быть предварительно исследованы в среде моделирования, а затем включены в состав микроконтроллерной прошивки без принципиальной переработки [9, 10, 12, 13].

Следовательно, архитектура программы соответствует задаче переноса на платформу STM32 не только на концептуальном уровне, но и на уровне реальной организации проекта, структуры программных модулей и системы сборки. Вместе с тем архитектурная пригодность к переносу не должна отождествляться с полной экспериментальной проверкой на целевом устройстве. Последняя требует отдельного этапа аппаратной интеграции, натурной отладки и проверки работы программы в составе реального робототехнического комплекса [9, 13, 15].

На рисунке 23 показано соответствие архитектурных уровней программы компонентам её реализации на платформе STM32.

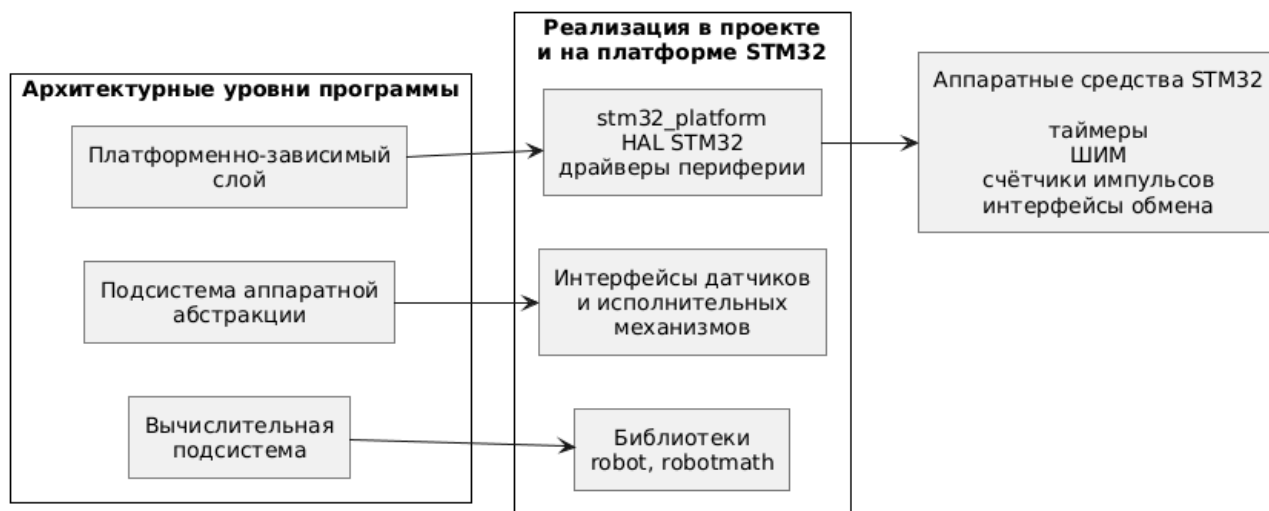


Рисунок 23 — Соответствие архитектурных уровней программы компонентам реализации на платформе STM32

4.5 Реализация взаимодействия с аппаратными ресурсами

При переносе программы управления на платформу STM32 основной объём адаптации сосредоточивается в реализации интерфейсов подсистемы аппаратной абстракции средствами периферийных устройств микроконтроллера. При этом подсистема управляющей логики и вычислительная подсистема управления сохраняются без принципиальных изменений. Изменению подлежит прежде всего платформенно-зависимая подсистема, обеспечивающая получение входных данных и передачу результатов вычисления исполнительным механизмам [10, 12, 20, 22].

Реализация интерфейсов датчиков перемещения естественным образом основывается на использовании аппаратных таймеров микроконтроллера, работающих в режиме счёта импульсов. Применение такого режима позволяет регистрировать изменение положения колёс по сигналам энкодеров и на этой

основе определять параметры их вращения, необходимые для последующего вычисления линейной и угловой скорости платформы. Использование аппаратных средств счёта при этом снижает нагрузку на центральное вычислительное ядро и повышает точность регистрации импульсных сигналов [20, 21, 22].

Управление приводами реализуется посредством таймеров, формирующих сигналы широтно-импульсной модуляции. Изменяя коэффициент заполнения сигнала, программа получает возможность изменять величину управляющего воздействия, подаваемого на двигатель. Тем самым микроконтроллерная платформа обеспечивает непосредственное преобразование вычисленного управляющего сигнала в форму, пригодную для воздействия на исполнительные механизмы ходовой части [20, 21, 22].

В случае использования инерциального измерительного модуля его подключение может быть организовано через стандартные последовательные интерфейсы обмена данными, такие как I²C или SPI. При этом принципиально важно, что логика использования информации от датчика в подсистеме управляющей логики и вычислительной подсистеме управления не изменяется. Замене подлежит только нижний уровень, отвечающий за чтение данных из конкретного устройства и их передачу через интерфейсы аппаратной абстракции [20, 21, 22].

Отдельное значение имеет организация периодического запуска алгоритма управления. Для этой цели могут использоваться системный таймер либо аппаратные таймеры общего назначения, обеспечивающие формирование событий через заданные интервалы времени. Именно эти средства определяют временную структуру работы программы и инициируют выполнение очередного шага дискретного цикла управления с требуемым периодом [7, 20, 21, 22].

Таким образом, при переходе к платформе STM32 изменяется не логика формирования управления как таковая, а способ её сопряжения с физическими датчиками, исполнительными механизмами и средствами синхронизации

микроконтроллера. Это подтверждает, что задача адаптации носит главным образом характер платформенной реализации нижнего уровня при сохранении разработанной архитектуры управления [10, 12, 22].

4.6 Организация цикла управления на микроконтроллере

Ключевым аспектом переноса программы управления на платформу STM32 является организация дискретного цикла работы в условиях ограниченных вычислительных ресурсов и требований к предсказуемости времени выполнения. Как и в моделируемой среде, общий алгоритм должен выполняться периодически с шагом дискретизации Δt , причём на каждом шаге последовательно выполняются обновление данных датчиков, формирование текущего состояния системы, вычисление управляющего воздействия и передача команд исполнительным механизмам [7, 9, 15].

С учётом архитектуры, рассмотренной в разделе 3.3, на одном шаге такого цикла компонент Robot сначала обеспечивает обновление информации от датчиков через подсистему аппаратной абстракции и нижележащую платформенно-зависимую подсистему. После этого Robot формирует текущее состояние платформы, которое используется компонентом RobotController совместно с командой движения для вычисления результирующего управляющего воздействия. Далее рассчитанное воздействие передаётся обратно в Robot, который обеспечивает его применение к исполнительному уровню через моторные обёртки и соответствующие интерфейсы нижнего уровня [10, 12, 22].

На микроконтроллерной платформе такой цикл может быть реализован двумя основными способами. Первый способ основан на использовании аппаратного таймера и механизма прерываний, при котором запуск очередного шага управления инициируется аппаратно через строго заданные интервалы времени. Второй способ состоит в выполнении управляющего алгоритма в основном цикле программы с контролем момента запуска по системному времени. С инженерной точки зрения более предпочтительным является первый подход, поскольку он обеспечивает более высокую точность периодичности и в

большей степени соответствует требованиям к детерминированности работы системы управления [7, 15, 20].

Принципиальным условием корректного функционирования системы является выполнение всех этапов одного шага управления за время, меньшее периода дискретизации. Это условие может быть записано в следующем виде [7, 15]:

$$T_{\text{считывание}} + T_{\text{оценка}} + T_{\text{управление}} + T_{\text{применение}} < \Delta t$$

Данное соотношение выражает требование временной реализуемости алгоритма на целевой платформе. Если суммарное время выполнения операций превышает период дискретизации, система теряет требуемую периодичность работы, что приводит к нарушению режима управления и может ухудшить качество регулирования [7].

С учётом структуры разработанной программы можно предполагать, что выполнение указанного условия является достижимым для выбранной микроконтроллерной платформы. Это обусловлено тем, что алгоритм управления включает вычислительные процедуры сравнительно невысокой сложности: фильтрацию первого порядка, кинематические преобразования и расчёт управляющих воздействий по законам регулирования, не требующим значительных вычислительных затрат. Вместе с тем данный вывод носит предварительный характер, поскольку его окончательное подтверждение возможно только по результатам измерения времени выполнения на реальном микроконтроллере [15, 21].

Следовательно, по принципам организации времени выполнения разработанная программа соответствует типовой структуре встраиваемой системы управления, в которой алгоритм функционирует как периодический дискретный процесс. Однако экспериментальная проверка временных характеристик, устойчивости периода запуска и фактической загрузки вычислительных ресурсов относится к следующему этапу практической интеграции программы на целевой аппаратной платформе [7, 9, 15]. На рисунке

24 представлена организация дискретного цикла управления на платформе STM32.



Рисунок 24 — Организация дискретного цикла управления на платформе STM32

4.7 Особенности реализации и ограничения перехода от моделирования к целевой платформе

При переносе разработанной программы управления на микроконтроллерную платформу STM32 необходимо учитывать ряд особенностей, характерных для встраиваемых вычислительных систем. Одной из наиболее существенных является целесообразность использования статического распределения памяти и отказа от динамического выделения памяти в процессе штатного функционирования программы. Для систем управления такое требование имеет принципиальное значение, поскольку позволяет снизить неопределённость времени выполнения операций, повысить предсказуемость поведения программного обеспечения и уменьшить вероятность ошибок, связанных с управлением памятью [9, 15].

Другой важной особенностью является ограниченность вычислительных и аппаратных ресурсов микроконтроллера по сравнению со средой исполнения на персональном компьютере. Хотя применяемые в работе алгоритмы не относятся к числу вычислительно сложных, их работоспособность на целевой платформе должна подтверждаться не только теоретически, но и экспериментально. Это относится как к времени выполнения отдельных вычислительных процедур, так и к соблюдению требуемой периодичности управляющего цикла. По этой причине архитектурные решения, основанные на использовании абстракций, интерфейсов и многоуровневого разделения компонентов, должны оцениваться не только с точки зрения удобства разработки и сопровождаемости кода, но и с точки зрения их влияния на быстродействие и детерминированность работы системы на микроконтроллере [9, 10, 12, 15, 21].

При переходе от программного моделирования к реальной аппаратной платформе необходимо учитывать и воздействие физических факторов, которые в моделируемой среде либо отсутствуют, либо представлены в упрощённом виде. К таким факторам относятся шумы и погрешности реальных датчиков, задержки обмена с периферийными устройствами, нелинейность характеристик исполнительных механизмов, проскальзывание ведущих колёс, влияние типа поверхности движения, а также изменение параметров системы при колебаниях напряжения питания. Наличие указанных факторов может привести к расхождению между расчётным и фактическим поведением платформы и, как следствие, потребовать дополнительной настройки коэффициентов регуляторов, уточнения параметров математической модели и корректировки отдельных элементов алгоритма управления после переноса программы на целевое устройство [1, 6, 8, 14, 20].

В связи с этим результаты программного моделирования следует рассматривать прежде всего как подтверждение корректности выбранной архитектурной схемы, алгоритмической структуры и вычислительной логики системы управления. Вместе с тем такие результаты не могут в полной мере

заменить этап аппаратной отладки и натурной проверки. С инженерной точки зрения это соответствует естественной последовательности разработки: на начальном этапе подтверждается корректность функционирования программы в контролируемой среде, после чего выполняются перенос на целевую платформу, аппаратная интеграция и экспериментальная проверка работы системы в условиях, приближённых к реальным [10, 12, 13].

4.8 Выводы

В данной главе показано, что разработанная программа управления ориентирована на использование в составе системы на базе микроконтроллера STM32 не только на уровне постановки задачи, но и на уровне организации проекта, архитектуры программного обеспечения и системы сборки [9, 15, 17, 21]. В структуре проекта предусмотрен отдельный раздел, связанный с прошивкой целевой платформы, выделена конфигурация сборки под ARM-архитектуру, а разработанные вычислительные модули включаются в состав исполняемого файла прошивки.

Принятое архитектурное разделение на подсистему управляющей логики, вычислительную подсистему управления, подсистему аппаратной абстракции и платформенно-зависимую подсистему делает перенос программы на STM32 технически обоснованным и практически реализуемым. В рамках такой структуры адаптация к микроконтроллерной платформе сводится главным образом к реализации нижнего уровня, обеспечивающего работу с энкодерами, исполнительными механизмами, датчиками и средствами временной синхронизации с использованием аппаратных ресурсов STM32.

Вместе с тем в работе необходимо различать готовность программного проекта к сборке под целевую архитектуру и полную экспериментальную верификацию на реальном устройстве. Если первая задача в разработанном проекте уже поддержана за счёт соответствующей структуры исходного кода и конфигурации сборки, то вторая требует отдельного этапа аппаратной

интеграции, измерения временных характеристик, настройки параметров системы управления и проведения натурных испытаний.

Таким образом, разработанная программа может рассматриваться как программная основа для последующей интеграции в систему управления ходовой частью четырёхколёсного робота на базе STM32. Полученные результаты подтверждают практическую направленность выполненной разработки, а также переносимость реализованной вычислительной логики на целевую встраиваемую платформу.

Заключение

В ходе выполнения выпускной квалификационной работы разработана программа управления ходовой частью четырёхколёсного мобильного робота с дифференциальной схемой привода, ориентированная на последующий перенос на микроконтроллерную платформу STM32.

В работе мобильная платформа рассмотрена как объект автоматического управления, выделены её конструктивные особенности и параметры движения, существенные для построения системы управления. На этой основе построена математическая модель объекта, сформулирована задача управления и разработан закон управления, обеспечивающий формирование управляющих воздействий по заданным линейной и угловой скоростям платформы.

В программной части работы спроектирована архитектура системы управления, основанная на разделении вычислительной логики, подсистемы аппаратной абстракции и платформенно-зависимых компонентов. Реализован дискретный цикл управления, включающий получение данных датчиков, формирование текущего состояния платформы, вычисление управляющего воздействия и его передачу исполнительным механизмам.

Для подтверждения корректности разработанной программы выполнено модульное тестирование основных программных компонентов и проведены испытания полного замкнутого контура управления в среде моделирования. Полученные результаты показывают, что система управления обеспечивает устойчивое воспроизведение заданных режимов движения и работоспособна в составе полного контура. При этом для длительных и составных траекторий выявляется накопление ошибки положения, что соответствует принятой структуре скоростного управления.

Отдельно рассмотрены особенности переноса разработанной программы на платформу STM32. Показано, что структура проекта, организация сборки и принятая архитектура обеспечивают возможность такой адаптации без принципиальной переработки вычислительного ядра системы. При этом полная экспериментальная проверка на реальной аппаратной платформе требует отдельного этапа аппаратной интеграции и натурной отладки.

Таким образом, поставленная в работе цель достигнута: разработана программа управления ходовой частью мобильного робота, обеспечивающая воспроизведение заданного режима движения платформы и допускающая последующий перенос на платформу STM32.

Перспективы дальнейшего развития работы связаны с переносом программы на целевую микроконтроллерную платформу, экспериментальной проверкой временных характеристик цикла управления, развитием механизмов ограничения исполнительных воздействий, расширением использования алгоритмов оценки состояния и введением внешнего позиционного контура управления.

Список источников

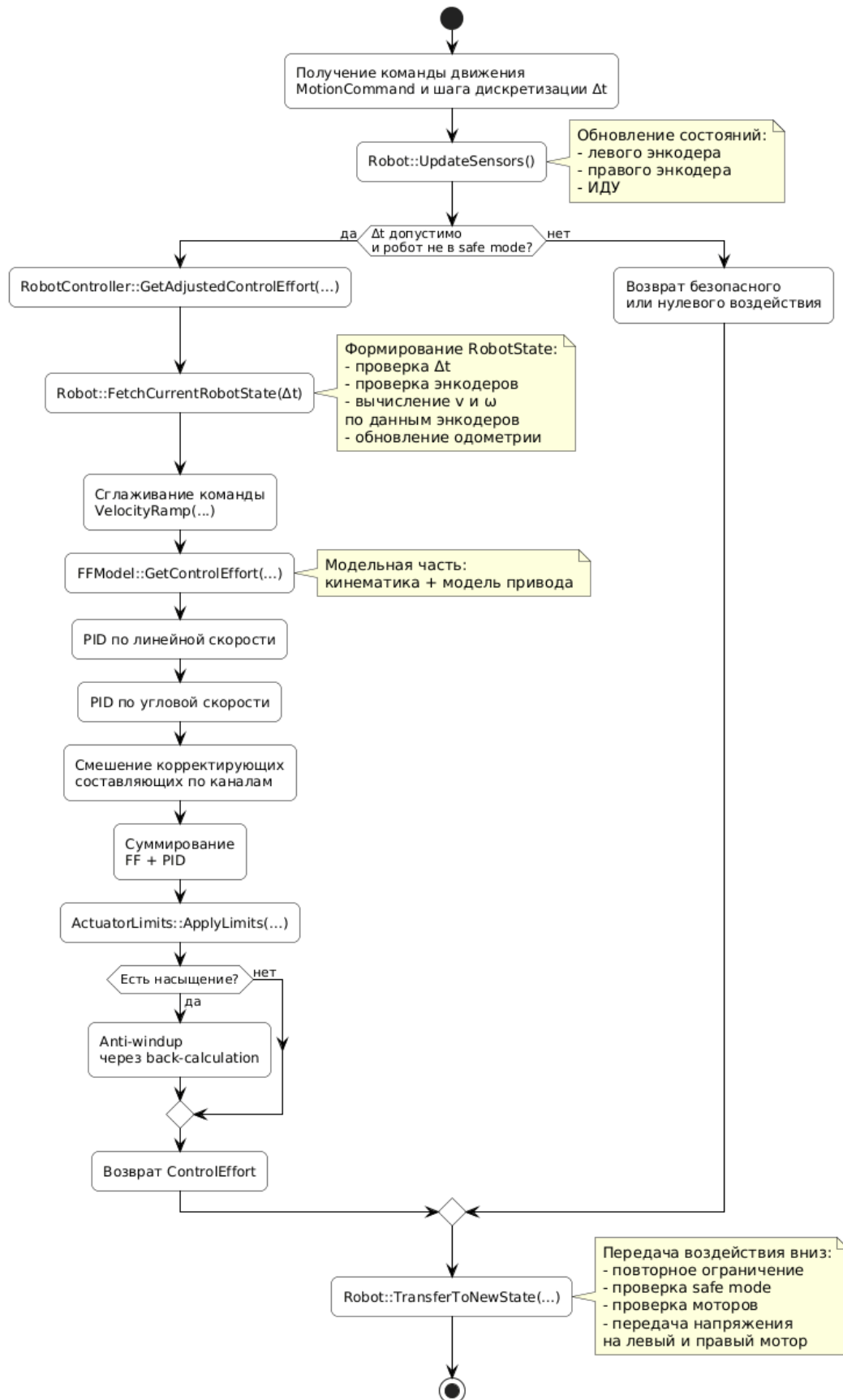
1. Бесекерский В. А., Попов Е. П. Теория систем автоматического управления. — 4-е изд., перераб. и доп. — СПб. : Профессия, 2007. — 747 с.
2. ГОСТ 19.301–79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению.
3. ГОСТ Р 7.0.5–2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления.
4. ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.
5. Давыдов О. И., Платонов А. К. Алгоритм управления дифференциальным приводом мобильного робота РБ-2 // Препринты ИПМ им. М. В. Келдыша. — 2015. — № 25. — 16 с.
6. Денисенко В. В. ПИД-регуляторы: принципы построения и модификации // Современные технологии автоматизации. — 2006. — № 4.
7. Иванов В. А., Ющенко А. С. Теория дискретных систем автоматического управления : учебное пособие для втузов / под ред. Е. П. Попова. — М. : Наука, 1983. — 335 с.
8. Кампион Г., Бастен Ж., Д’Андреа-Новель Б. Структурные свойства и классификация кинематических и динамических моделей колесных мобильных роботов // Нелинейная динамика. — 2011. — Т. 7, № 4. — С. 733–769.
9. Ключарёв А. А., Кочин К. А., Фоменкова А. А. Программирование микроконтроллеров STM32 : учебное пособие. — СПб. : ГУАП, 2023. — 196 с.
10. Липаев В. В. Программная инженерия. Методологические основы : учебник для вузов. — М. : ТЕИС, 2006. — 605 с.

11. Мартыненко Ю. Г. Управление движением мобильных колёсных роботов // Фундаментальная и прикладная математика. — 2005. — Т. 11, вып. 8. — С. 29–80.
12. Орлов С. А., Цилькер Б. Я. Технологии разработки программного обеспечения : учебник для вузов. — 4-е изд. — СПб. : Питер, 2012. — 608 с.
13. Оськин Д. А., Громашева О. С., Дьяченко М. Е. Модельно-ориентированный подход для автоматизации генерации программного С-кода для встраиваемых систем из модели MATLAB/Simulink // Современные наукоемкие технологии. — 2018. — № 10. — С. 92–97.
14. Усольцев А. А. Электрический привод : учебное пособие. — СПб. : НИУ ИТМО, 2012. — 238 с.
15. Федосов Ю. В., Шубин А. С. Cortex M4. Лабораторный практикум : учебно-методическое пособие. — СПб. : Университет ИТМО, 2020. — 66 с.
16. Юревич Е. И. Основы робототехники : учебник для студентов вузов. — Л. : Машиностроение, 1985. — 270 с.
17. CMake Documentation. Cross Compiling With CMake. — Электронный ресурс. — URL: <https://cmake.org/cmake/help/book/mastering-cmake/chapter/Cross%20Compiling%20With%20CMake.html>
18. Cyberbotics. Webots Documentation. Controller Programming. — Электронный ресурс. — URL: <https://cyberbotics.com/doc/guide/controller-programming>
19. GoogleTest. GoogleTest User's Guide. — Электронный ресурс. — URL: <https://google.github.io/googletest/>
20. STMicroelectronics. RM0383 Reference manual. STM32F411xC/E advanced Arm-based 32-bit MCUs. — Электронный ресурс. — URL: https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xc-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

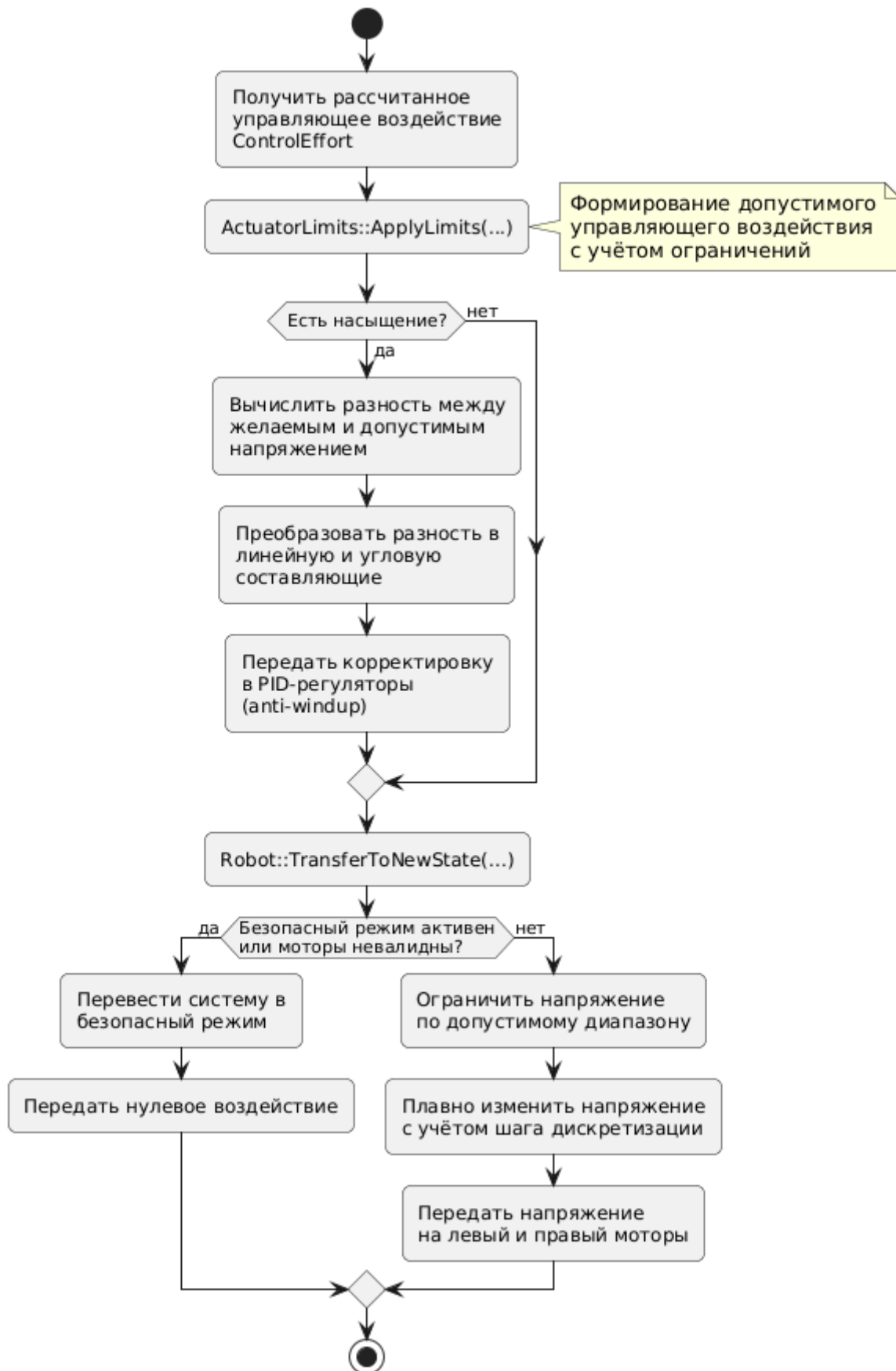
21. STMicroelectronics. STM32F411xC STM32F411xE Datasheet. DS10314. —
Электронный ресурс. — URL:
<https://www.st.com/resource/en/datasheet/stm32f411ce.pdf>
22. STMicroelectronics. UM1725. Description of STM32F4 HAL and low-layer
drivers. — Электронный ресурс. — URL:
https://www.st.com/resource/en/user_manual/um1725-description-of-stm32f4-hal-and-lowlayer-drivers-stmicroelectronics.pdf

Приложения

Приложение А



Приложение Б



Приложение В

